

PYTHON機器學習入門

UNIT 1 : CRASH COURSE ON PYTHON

授課教師：江尚瑀

OUTLINE

- Python基礎、Python資料結構

Python Basics

```
#print Hello Python 101  
print("Hello Python 101")  
Hello Python 101
```

11	int
21.213	float
"Hello Python 101"	str

total_min

142

/ 60

String
methods

Sequence
methods

Python Data Structures

```
L=["Michael Jackson", 10.1, 1982]
```

```
Ratings=(10, 9, 6, 5, 10, 8, 9, 6, 2)
```

```
album_set=set(album_list)
```

album_list

```
{"key1":1, "key2 ":"2","key3":[3,3,3], "key4):(4,4,4), ('key5'):5}
```

- Python程式設計邏輯基礎架構、Python如何處理資料

Python Programming Fundamentals



```
if (age>18):  
else:  
elseif (age==18)
```

```
for i in range(0,5):  
while(squares[i]=='orange'):
```

len

sum

sorted

```
def function(a):  
    """add 1 to a"""  
    b=a+1;  
    print(a, "+1 =",b)  
    return b
```

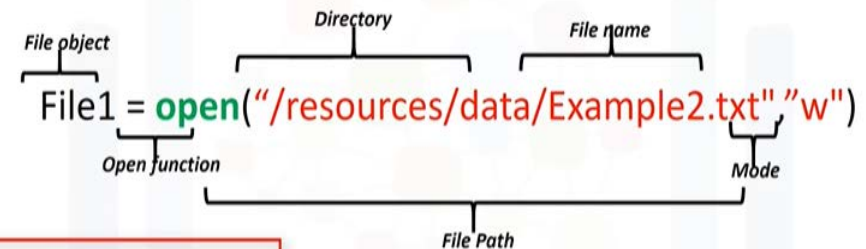
Name of Object

Name of Class

RedCircle = Circle(10, "red")

Object Constructor Attributes

Working with Data in Python



```
import pandas
```

PYTHON INTRO/ENV



PART I

WHY IS PYTHON ?

TIOBE Index for 2020 & 2021

Jan 2021	Jan 2020	Change	Programming Language	Ratings	Change
1	2	▲	C	17.38%	+1.61%
2	1	▼	Java	11.96%	-4.93%
3	3		Python	11.72%	+2.01%
4	4		C++	7.56%	+1.99%
5	5		C#	3.95%	-1.40%

Feb 2021	Feb 2020	Change	Programming Language	Ratings	Change
1	2	▲	C	16.34%	-0.43%
2	1	▼	Java	11.29%	-6.07%
3	3		Python	10.86%	+1.52%
4	4		C++	6.88%	+0.71%
5	5		C#	4.44%	-1.48%
6	6		Visual Basic	4.33%	-1.53%
7	7		JavaScript	2.27%	+0.21%
8	8		PHP	1.75%	-0.27%
9	9		SQL	1.72%	+0.20%
10	12	▲	Assembly language	1.65%	+0.54%

WHY IS PYTHON ?

- Python被列為嵌入式語言 (embedded language)
- 針對統計和機器學習，Python模組更為靈活多樣性
- 所有Machine Learning 軟體框架中都支援 Python

<https://venkatarangan.com/blog/2020/07/the-top-programming-languages-in-2020-by-ieee-spectrum/>

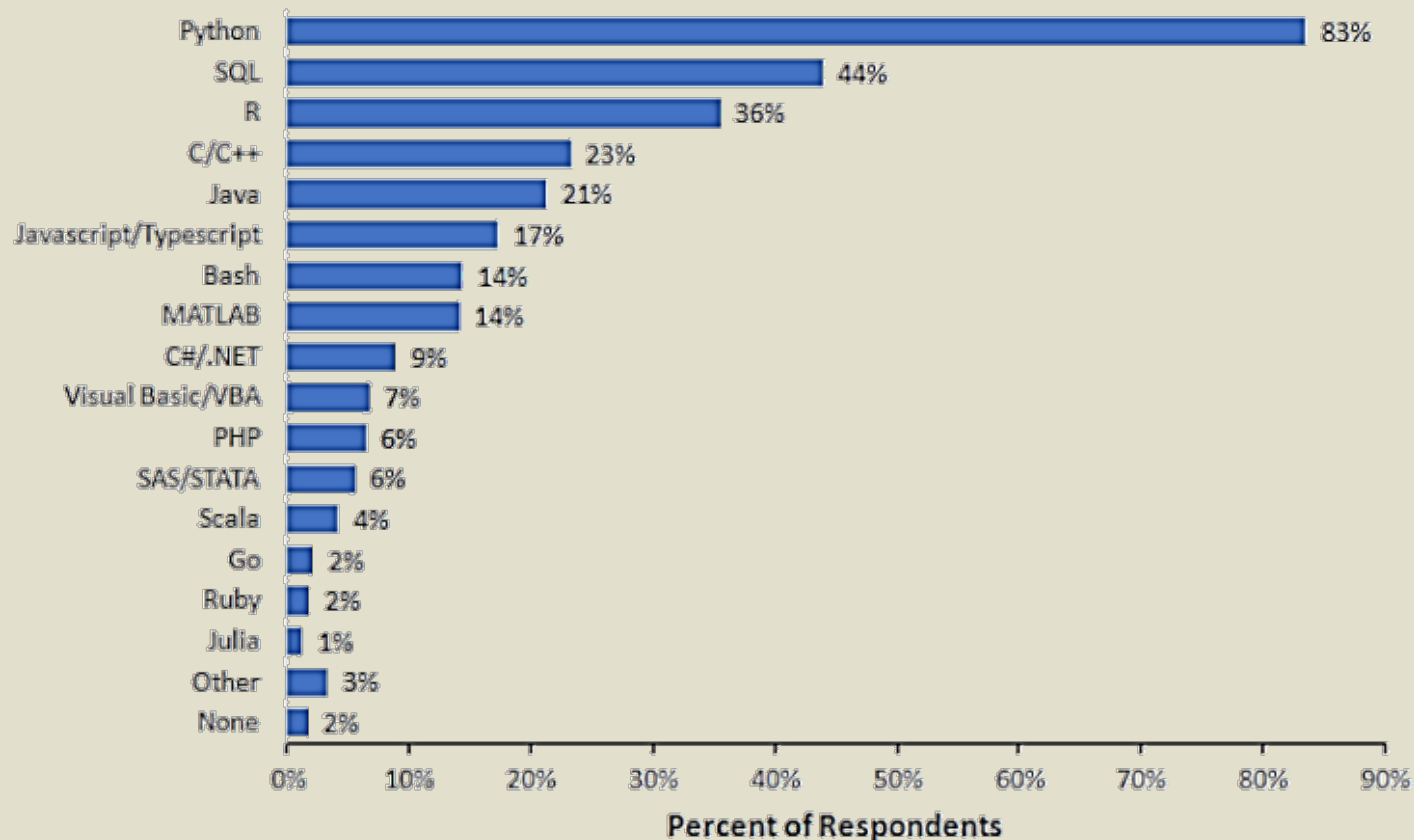
2020 IEEE Spectrum Top Programming Language

Language Rank	Types	Spectrum Ranking
1. Python	 	100.0
2. C	  	99.7
3. Java	  	99.4
4. C++	  	97.2
5. C#	  	88.6
6. R		88.1
7. JavaScript	 	85.5
8. PHP		81.4
9. Go	 	76.1
10. Swift	 	75.3
11. Arduino		73.0
12. Ruby	 	72.4
13. Assembly		72.1
14. Scala	 	68.3
15. Matlab		68.0
16. HTML		67.0
17. Shell		66.3
18. Perl	 	57.6
19. Visual Basic		55.4
20. Cuda		53.9

WHY IS PYTHON ?

Kaggle

What programming language do you use on a regular basis?



Note: Data are from the 2018 Kaggle Machine Learning and Data Science Survey. You can learn more about the study here: <http://www.kaggle.com/kaggle/kaggle-survey-2018>. A total of 18827 respondents answered the question.

HISTORY OF PYTHON

- 創始人

荷蘭 - 吉多·范羅蘇姆 (**Gudio van Rossum**)

- 誕生於**1989年12月Rossum**為排解聖誕節的無聊即開發一個新的指令碼解釋程式來打發時間
- 最初發布於**1991年**，自**ABC**程式語言衍生出來，並用熱愛的電視影集《蒙提·派森的飛行馬戲團》 (**Monty Python's Flying Circus**)最為發想替**Python**命名。



THE ZEN OF PYTHON

**There should be one
– and preferably only one –
obvious way to do it.**



「優雅」、「明確」、「簡單」

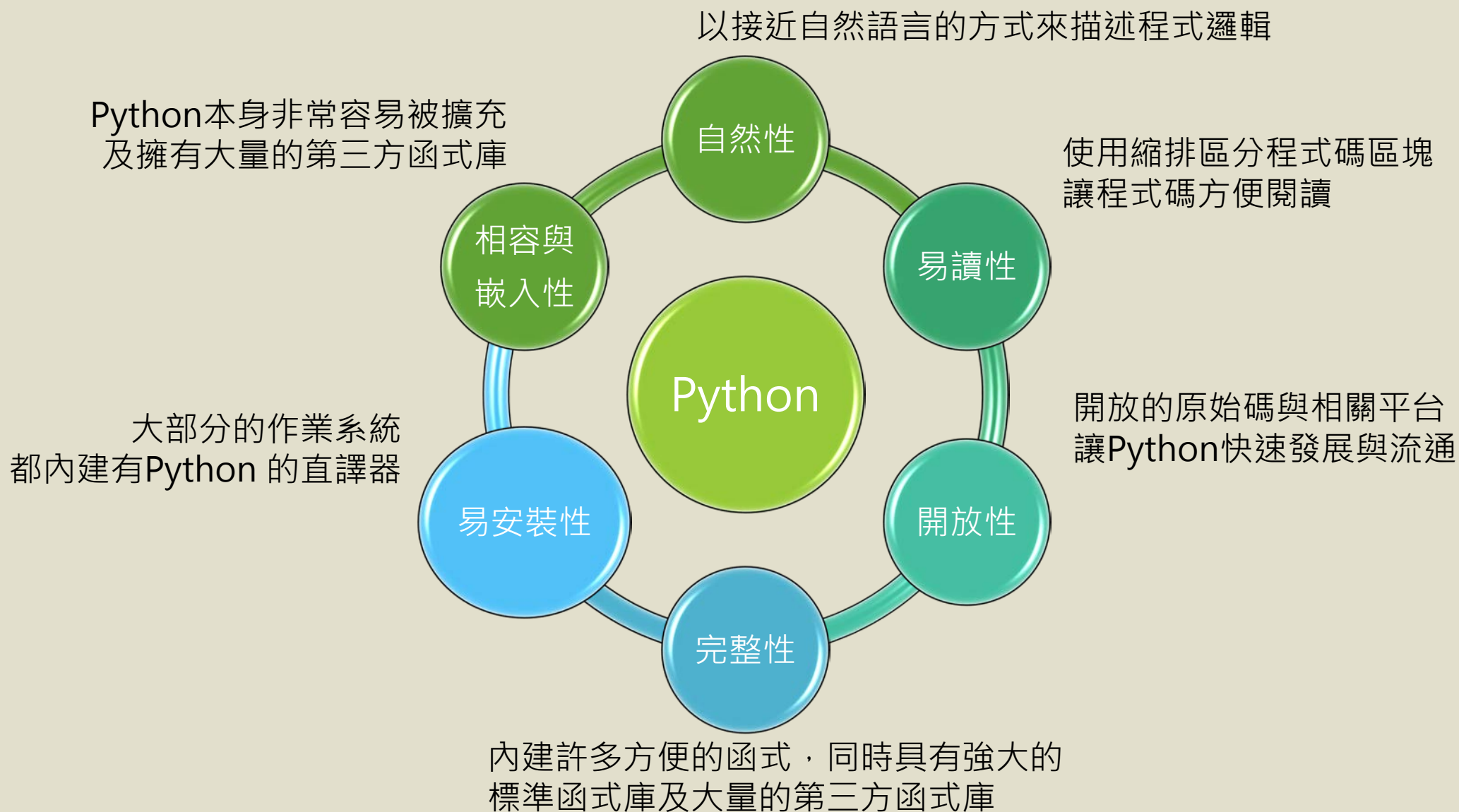
WHAT IS PYTHON?

- Python是完全物件導向的語言 (Object Oriented)
函式、模組、數字、字串都是物件
- 完全支援繼承、重載、衍生、多繼承
有益於增強原始碼的複用性
- 直譯式語言Interpreted language
有逐行執行的特性，可以即時反應程式碼的執行成果
- Python被分類為「腳本語言」(script language)
高階動態程式語言
- 套件豐富且安裝便利
在各領域皆有應用套件積極的被發表，不僅開發社群強大，非傳統資訊工程領域的使用者也非常多
- 膠水特性 (glue language)
容易跟其他語言進行整合封裝，或是被其他乘式語言調用。同時現在有很多專業應用軟體也支援Python操作

WHAT IS PYTHON?

- Python is an **easy** to learn, **powerful** programming language. It has efficient **high-level data structures** and a simple but effective approach to **object-oriented** programming.
- Python's **elegant syntax** and **dynamic typing**, together with its **interpreted** nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.
- <https://www.python.org/>

PYTHON 語言特性



PYTHON USERS



概述與環境配置

- python Anaconda安裝與環境設定
- Anaconda套件
- Anaconda安裝
- Conda語法

PYTHON ANACONDA安裝與環境設定

- Anaconda ， 蟒蛇(去一马 `)
- 包含了眾多流行的科學、數學、工程、數據分析的 Python包，完全開源和免費，額外的加速、優化是收費的，但對於學術用途可以申請免費的 License 。
- 全平台支持：Linux、Windows、Mac5.
- 支持 Python 2.6、2.7、3.4-3.8，可自由切換
- 內帶spyder 編譯器與jupyter notebook 環境
(舊版的ipython notebook)
- 優點：省時！
一鍵安裝完90%一般人這一輩子會用到的Python套件，剩下的再用pip install個別去安裝即可
- 缺點：占空間
包含了一堆用不到的Python的套件(可安裝另一種miniconda)
- 另外，安裝完Anaconda之後你會獲得一個叫Conda的指令可以用，可以讓你用conda管理不同Python的版本以及套件。

ANACONDA套件

NOW PLAY WITH THE WORLD'S MOST AWESOME DATA SCIENCE PACKAGES

Packaged included in Anaconda 4+, or get with "conda install PACKAGENAME"

1. NumPy | numpy.org

N-dimensional array for numerical computation

2. SciPy | scipy.org

Collection of numerical algorithms and toolboxes, including signal processing and optimization

3. Matplotlib | matplotlib.org

Plotting library for Python

4. Pandas | pandas.pydata.org

Powerful Python data analysis toolkit

5. Seaborn | stanford.edu/~mwaskom/software/seaborn/

Statistical data visualization

6. Bokeh | bokeh.pydata.org

Interactive web visualization library

7. SciKit-Learn | scikit-learn.org/stable

Python modules for machine learning and data mining

8. NLTK | nltk.org

Natural language toolkit

9. Notebook | jupyter.org

Web-based interactive computational environment combines code execution, rich text, mathematics, plots and rich media

10. R essentials | conda.pydata.org/docs/r-with-conda.html

R with 80+ of the most used R packages for data science
"conda install -c r r-essentials"

(以上為Anaconda內含的重要套件，完整套件可點此[參考](#))

CONDA是什麼？

- 如果慣用pip的人對於conda的指令一定也可以馬上上手，因為它和pip指令非常非常的相似
- Conda是套件管理系統，也可用來建立虛擬環境，不過因為Anaconda本身專注於數據分析，所以我們會使用到的像是pandas、NumPy、Scipy的python package在安裝完成時就已經包含在裡面不需要另外安裝了
- 記得不管要做什麼, 先確認一下環境的 conda 是最新版、資訊也是最新的, 所以都先 update conda 自己。

```
conda update conda
```

CONDA VS. PIP

	Conda	pip
依賴項檢查	① 列出所需其他依賴包。 ② 安裝包時自動安裝其依賴項。 ③ 可以便捷地在包的不同版本中自由切換。	① 不一定會展示所需其他依賴包。 ② 安裝包時或許會直接忽略依賴項而安裝，僅在結果中提示錯誤。
環境管理	比較方便地在不同環境之間進行切換，環境管理較為簡單。	維護多個環境難度較大。
對系統本身python影響	在系統自帶Python中包的更新/回退版本/卸載將影響其他程式。	不會影響系統自帶Python。
適用語言	僅適用於Python。	適用於Python, R, Ruby, Lua, Scala, Java, JavaScript, C/C++, FORTRAN。

ANACONDA 3.X 安裝

- 至官網下載區 <https://www.anaconda.com/download/> ，下載目前最新版之**Anaconda (Python 3.x version)**

Anaconda Installers

Windows 	MacOS 	Linux 
Python 3.8 64-Bit Graphical Installer (477 MB) 32-Bit Graphical Installer (409 MB)	Python 3.8 64-Bit Graphical Installer (440 MB) ☒ 64-Bit Command Line Installer (433 MB)	Python 3.8 64-Bit (x86) Installer (544 MB) 64-Bit (Power8 and Power9) Installer (285 MB) 64-Bit (AWS Graviton2 / ARM64) Installer (413 M) 64-bit (Linux on IBM Z & LinuxONE) Installer (292 M)

ANCONDA安裝

- Step01、Anaconda官方網站 <https://www.anaconda.com/download/>
- Step02、選擇作業系統與python版本(建議Python3.6 or 3.5)，可以先安裝3.7之後用指令修改
- Step03、點擊下載的檔案，依序下一步安裝
*請勾選Add path to the system PATH
- Step04、確認是否安裝成功
win+r，輸入cmd，>後輸入python
如下python3.7.3安裝成功

```
C:\Users\shangyu>python
Python 3.7.3 (default, Mar 27 2019, 17:13:21) [MSC v.1915 64 bit (AMD64)] :: Anaconda, Inc. on win32
```

ANCONDA安裝

另一個驗證方式，從Windows開始選單(Start menu)中選擇Anaconda Navigator，如果Navigator可以打開，表示您已成功安裝Anaconda。



透過 Anaconda Prompt 可以進入設好環境變數的終端機
其主要語法與命令提示字元cmd內，大致上一樣。

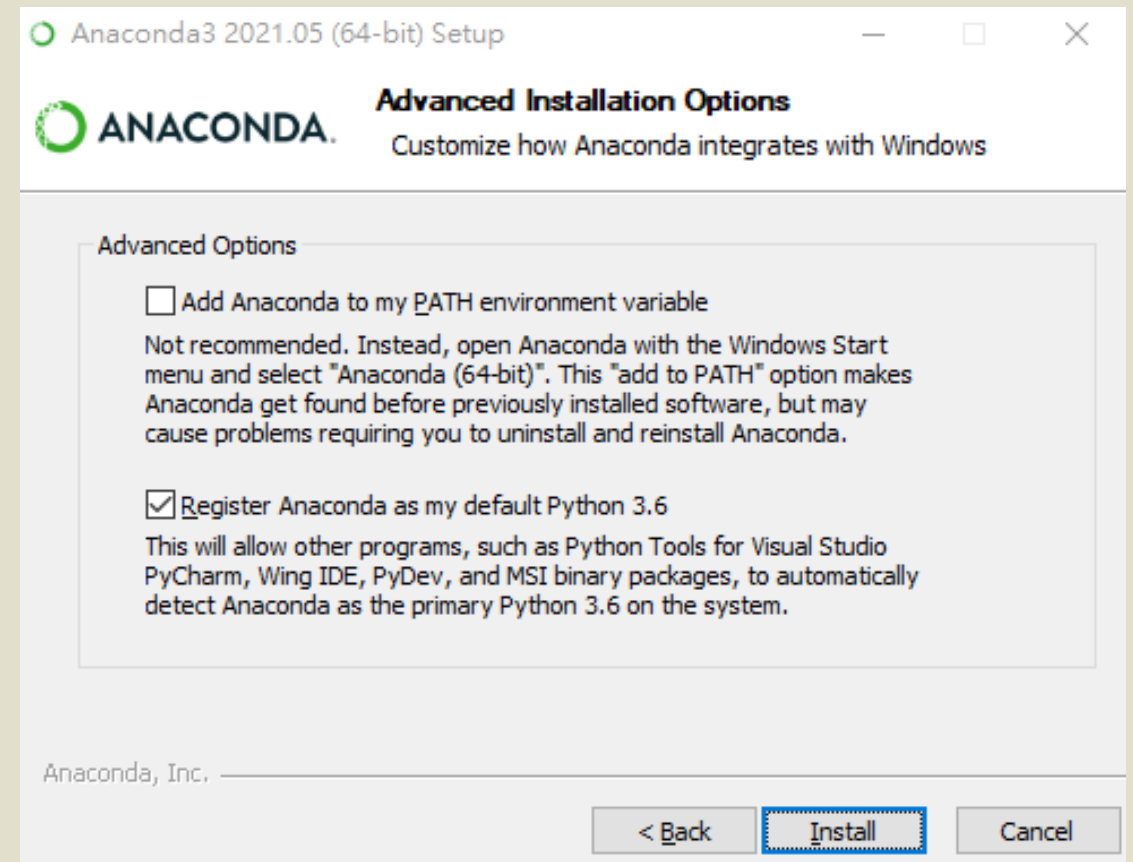
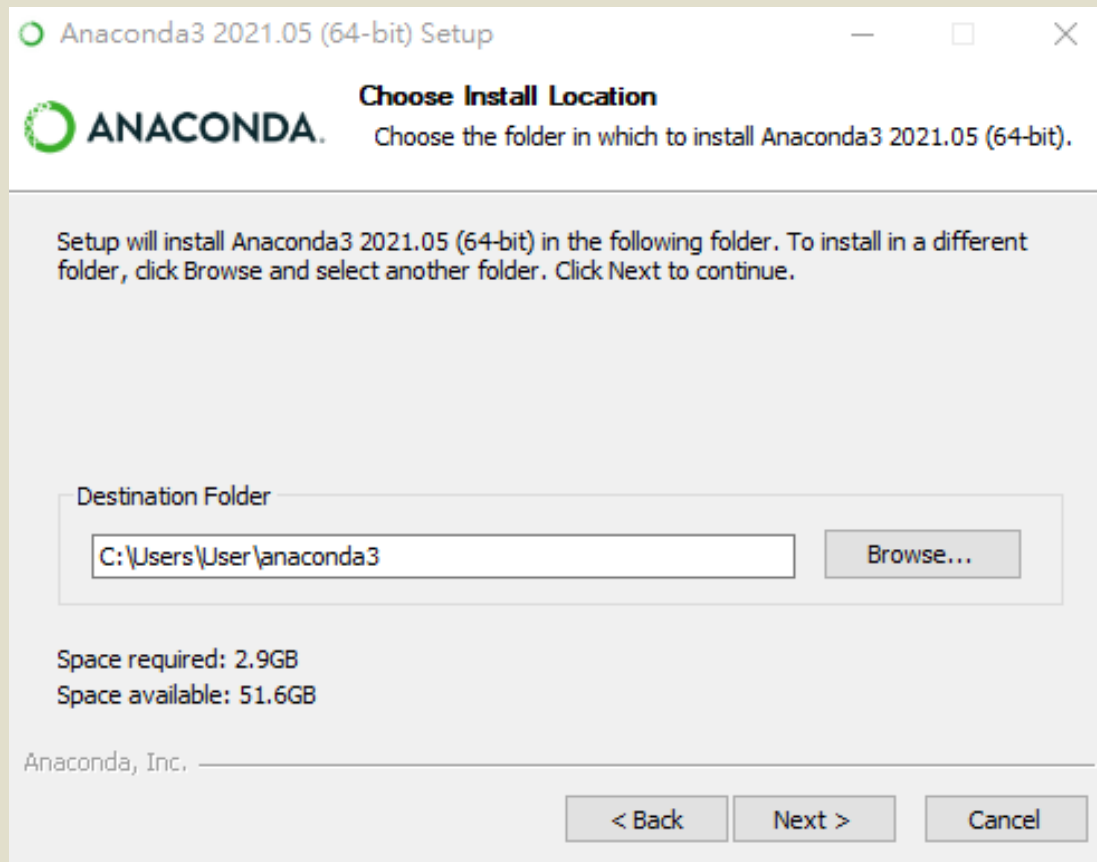
ANACONDA 3.X 安裝

- 下載完成後，雙點 Anaconda3 安裝程式開始進行安裝
- 依序點選「Next >」、「I Agree」、「Next >」



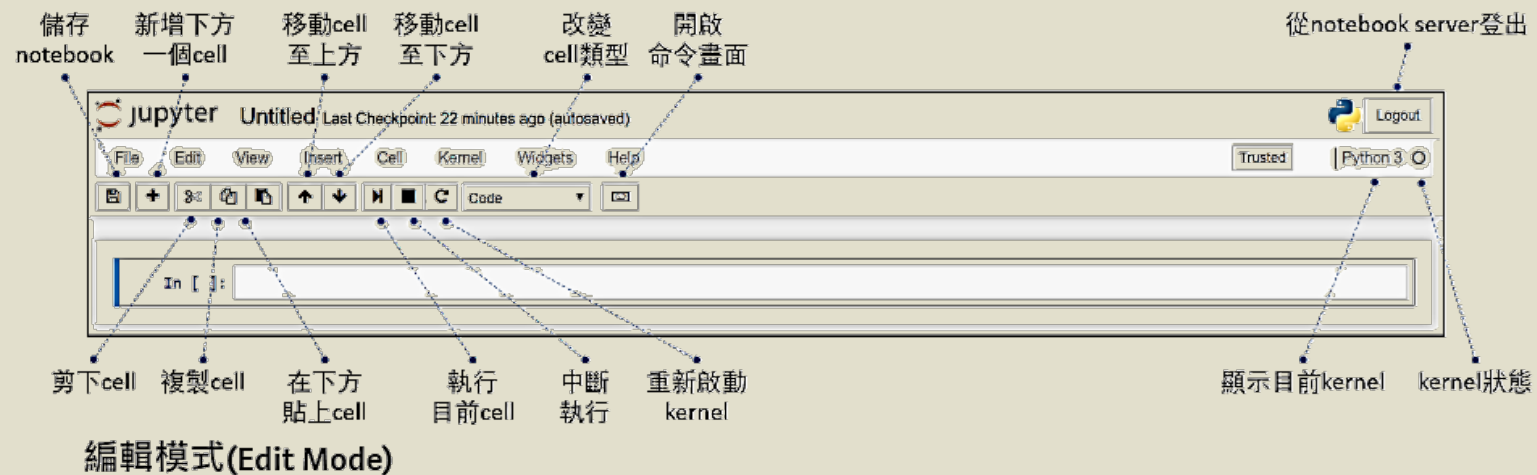
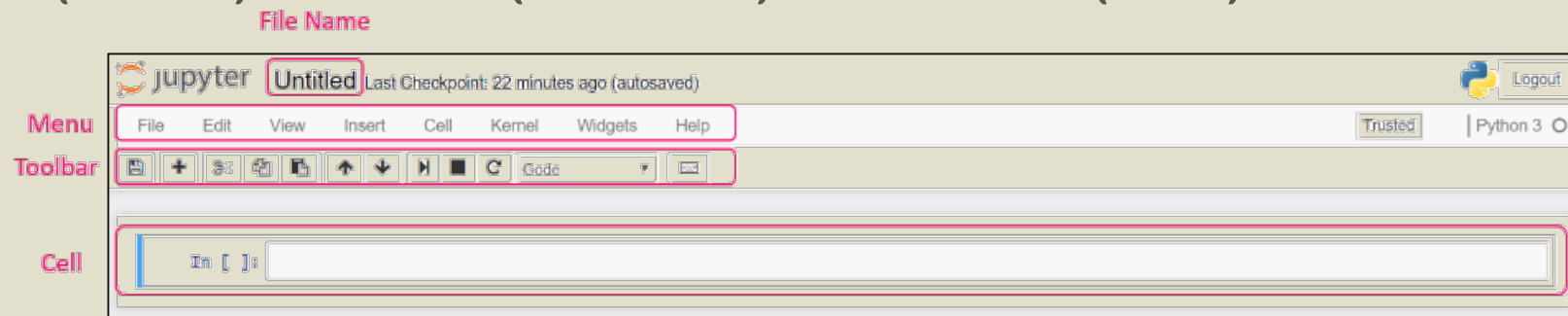
ANACONDA 3.X 安裝

- 確認安裝路徑後點選點選「Next >」，確認進階選項後點選「Install」，安裝成功後點選「Next >」、「Finish」完成安裝



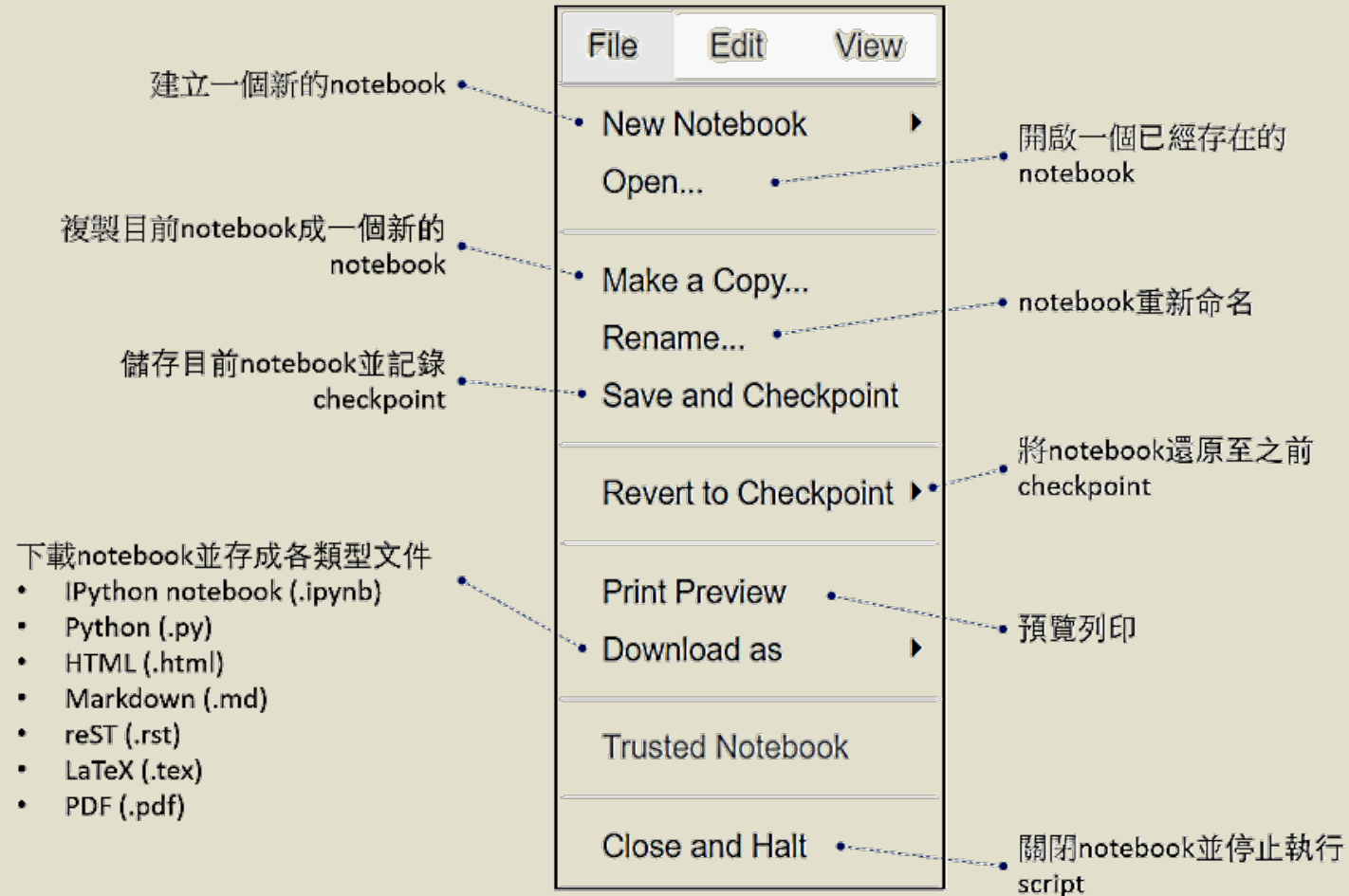
JUPYTER NOTEBOOK(1/7)

- Jupyter Notebook的編輯介面主要分為四部分：檔名(File Name)、主選單(Menu)、工具列(Toolbar)及編輯單元(Cell)。



JUPYTER NOTEBOOK(2/7)

- 儲存及載入(Saving/Loading)



JUPYTER NOTEBOOK(3/7)

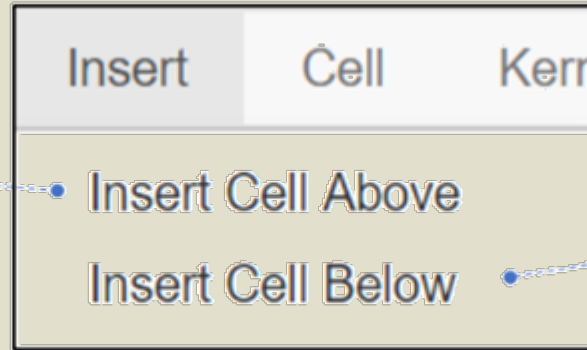
- 程式碼和文本是由3種基本cells類型所包裝起來：**Markdown cells**、**Code cells**及**Raw NBConvert cells**。



JUPYTER NOTEBOOK(4/7)

• Insert

在目前Cell上方
插入一個新Cell



在目前Cell下方
插入一個新Cell

• Executing

執行選定的Cell(s)

執行目前Cell(s)並且在下方插入
一個新的Cell

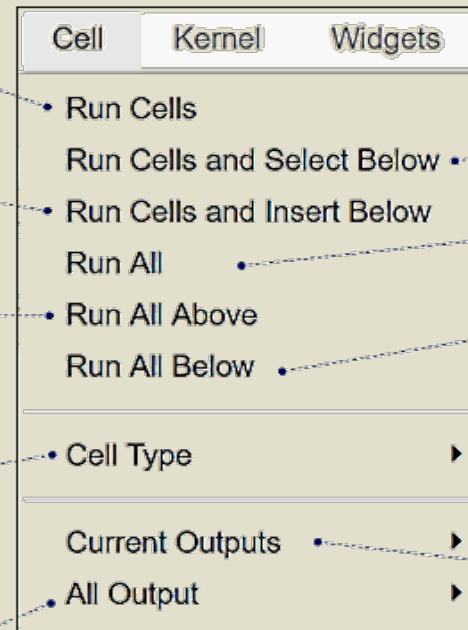
執行目前Cell上方所有的Cells

更改目前Cell的類型

- Code
- Markdown
- Raw NBConvert

對所有Cell輸出做下列動作

- Toggle(隱藏/顯示切換)
- Toggle Scrolling(上下切換)
- Clear(清除)



執行目前Cell(s)並且移動至下
方Cell

執行全部的Cells

執行目前Cell下方所有的Cells

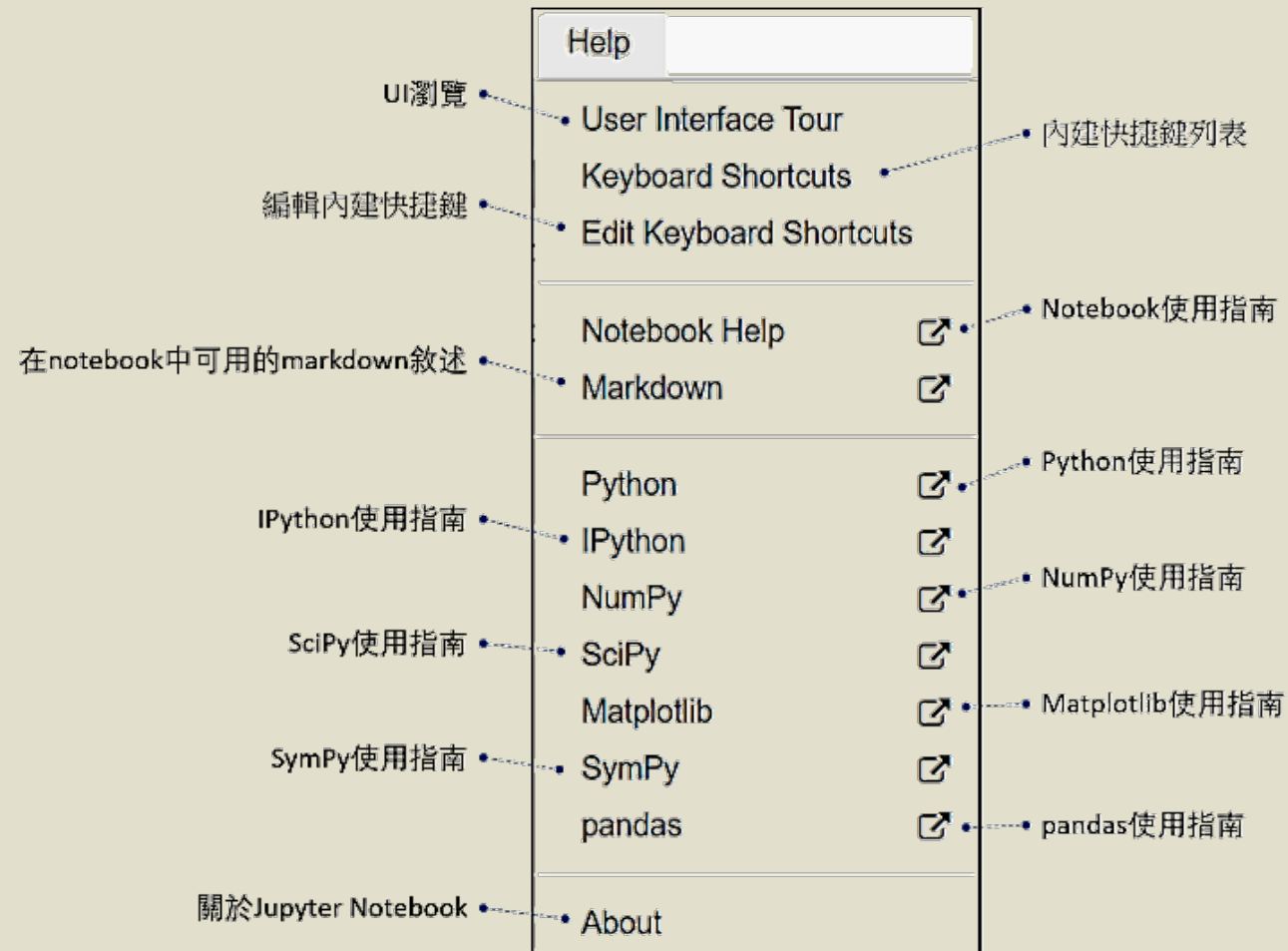
對目前Cell輸出做下列動作

- Toggle(隱藏/顯示切換)
- Toggle Scrolling(上下切換)
- Clear(清除)

JUPYTER NOTEBOOK(5/7)

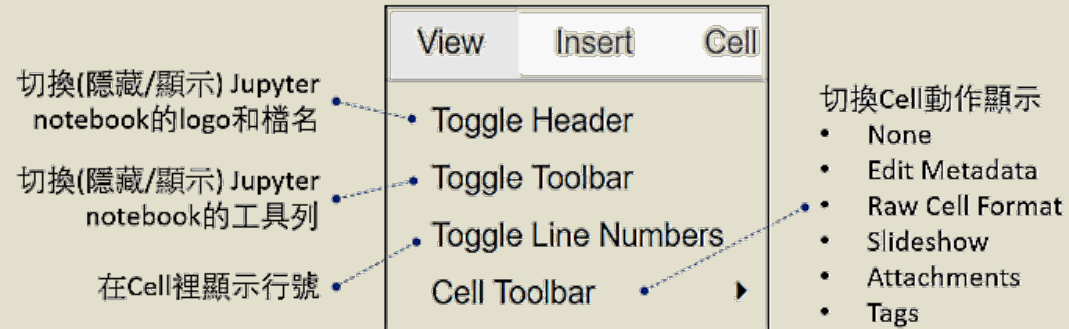
• Help

Jupyter Notebook提供了許多線上的使用指南，包括在**Data Science**中常用的**packages**。

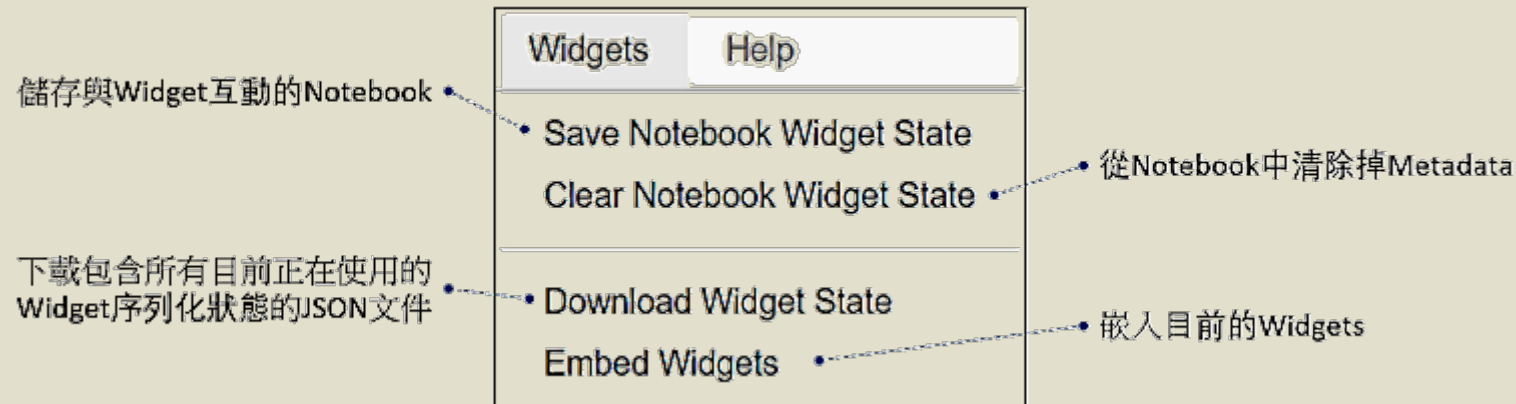


JUPYTER NOTEBOOK(6/7)

- View



- Widgets - 提供了可視覺化及控制數據變化的能力



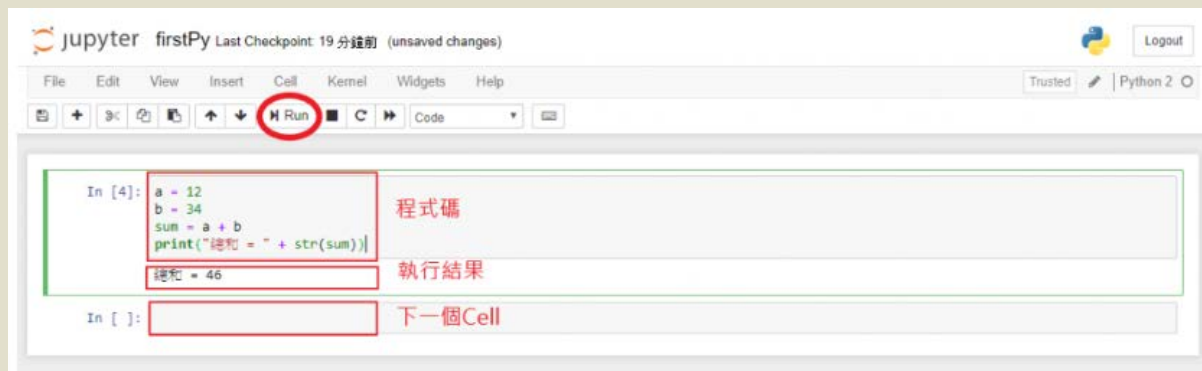
JUPYTER NOTEBOOK(7/7)

- 執行程式

- 1.按工具列的**Run**按鈕

- 2.按**Ctrl+Enter**

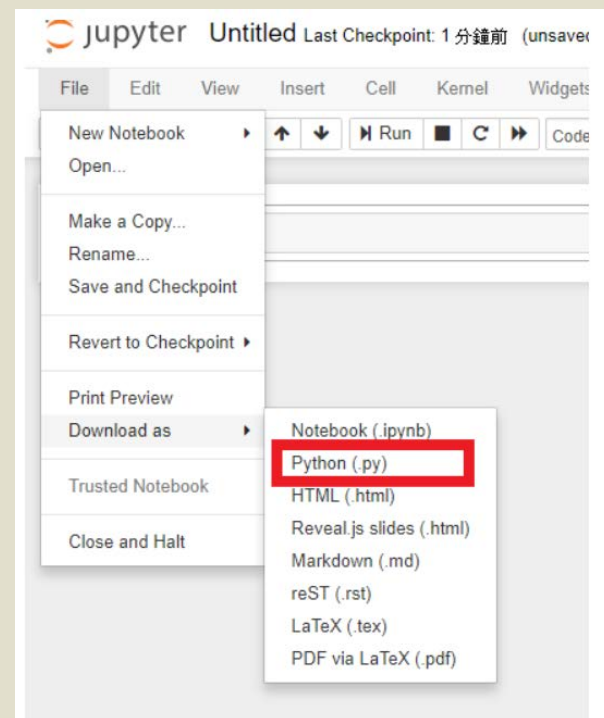
- 3.按**Shift+Enter**



- Jupyter Notebook

建立檔案的副檔名為「.ipynb」
如果要另外存成「.py」

- 列出所有的快捷鍵 **Esc+h**



A decorative graphic on the left side of the slide consisting of two parallel, wavy vertical lines. The inner line is a vibrant lime green, and the outer line is a light cream color. They extend from the top to the bottom of the frame.

PYTHON BASICS

PART 2

WHITE SPACE MATTERS!

- Whitespace is meaningful in python:
 - **Indentation defines block in python**
 - **Use 4-space indentation, and no tabs.**
 - Placement of newline ends a statement.
 - Semicolon ; is not required
 - No braces {} required to mark blocks of code.
 - Often a colon: appears at the start of a new block
 - Use \ when must go to next line prematurely
 - Single line comment #
 - Multi line comment ''' '''

INDENTATION DEFINES BLOCK IN PYTHON

- 程式碼的易讀性更高，請務必讓同一個生命週期的程式碼保持在所屬的縮排區塊裡面。

(註:尤其像是Python更是需要透過縮排來區分程式碼分工的區域)

<http://coffeeghost.net>

Quick Python Script Explanation for Programmers

Load other code modules.

Module name. This refers to 'os.py'

```
import os
```

The name "main" is just a convention, not a requirement. See the very bottom of this script.

```
def main():
```

print statements. Also, there are no semicolons at the end of the line.

```
    print 'Hello world!'
```

No curly braces! Instead, the block starts when you add 4 spaces of indentation. (or any other amount, but 4 spaces is the standard)

```
    print "This is Alice's greeting."
    print 'This is Bob's greeting.'
```

I prefer single-quotes, but either is fine. Either way, you don't have to escape the other kind of quote inside the string.

```
    foo(5, 10)
```

Function call.

```
    print '=' * 10
```

String replication. Evaluates to '====='

```
    print 'Current working directory is ' + os.getcwd()
```

String concatenation.

```
    counter = 0
    counter += 1
```

Variables MUST be instantiated first.

Call a function in the os module.

```
food = ['apples', 'oranges', 'cats']
```

Lists can contain different data types in the same list, including other lists.

```
for i in food:
```

For loop. "i" takes on each value in the list "food" in order.

```
    print 'I like to eat ' + i
```

One-line block. When the indentation goes back down, the block has ended.

```
print 'Count to ten:'
```

The range() function returns a list like [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]. Don't forget the colon at the end!

```
for i in range(10):
```

Function definition. Don't forget the colon at the end.

```
    print i
```

```
def foo(param1, secondParam):
```

String interpolation works basically the same way as it does in C.

```
    res = param1 + secondParam
```

```
    print '%s plus %s is equal to %s' % (param1, secondParam, res)
```

The comparison operators are the same as C.

```
    if res < 50:
```

```
        print 'foo'
```

```
    elif (res >= 50) and (param1 == 42) or (secondParam == 24):
```

Boolean operators are words, not && and ||.

```
        print 'bar'
```

```
    else:
```

Colors come after def, for, while, if, elif, and else statements.

```
        print 'moc'
```

```
    return res
```

Comments.

```
    # This is a one-line comment.
```

```
    '''A multi-line string, but can also be a multi-line comment.'''
```

Multi-line strings don't affect block indentation, though, only the indentation at the START of the statement or expression.

```
if __name__ == '__main__':
```

We put a call to main() at the bottom so that each def statement is executed by the time we call main(). This script's __name__ variable has the value '__main__' only when the script is run, not imported. With this check, the main() function won't run if another script imports this script.

```
    main()
```

COMMENT IN PYTHON

- **Single line comment**

#這是一行單行註解，當程式執行到這一行時會自動跳過。

- **Multiple line comment**

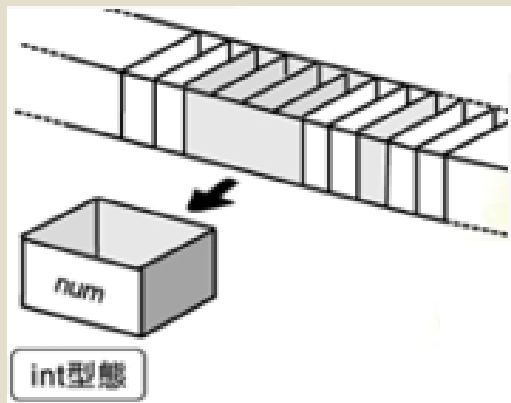
'''

若是想要描述的筆記內容比較多，
也可以透過一對連續的"三個單引號"作為多行註解。
我們可以把它便利且清楚地用換行的方式做處理。

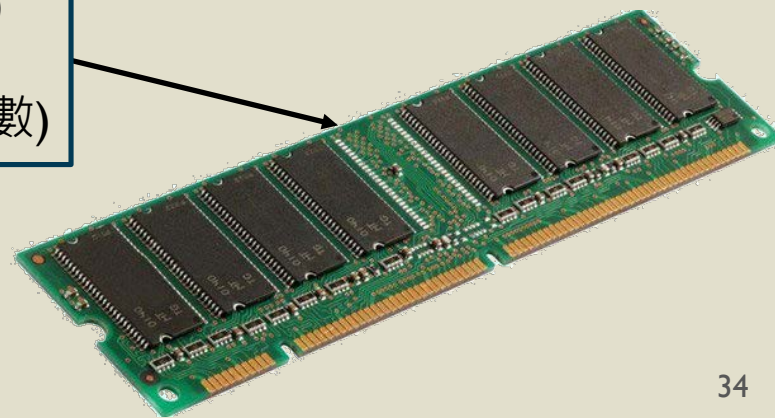
'''

變數 (VARIABLE)

- 在電腦裡，所有資訊都會放在**記憶體 (Random Access Memory, RAM)** 裡面，然後才進行執行與存取。
- 在程式語言中使用各種資訊或資料型態，都必須要跟系統取得**記憶體空間**。
- 使用變數就是一種跟系統要記憶體空間的動作。



Shelly (字串)
123 (整數)
123.45 (浮點數)

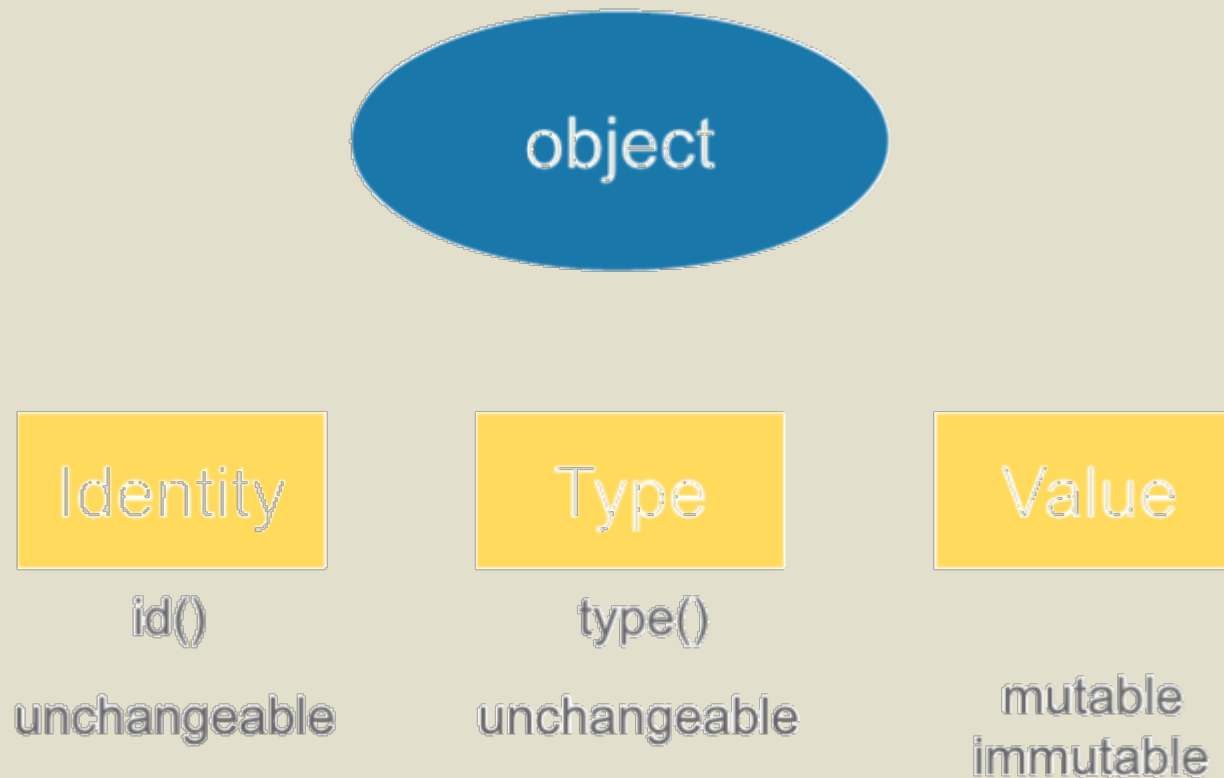


變數 (VARIABLE)

- 在**Python** 裡面的所有東西都被稱為物件 (**Object**)
- 物件就好像是生活中各式各樣的東西，如桌椅、水、人、交通工具等等，是一種統稱。
- 藉由**變數**來代表**各種物件**，就好像學生在一個班級裡會有學號，以及人會有身分證字號是一樣的意思。
- 有時**不同的變數**可以**代表同一個物件**，就好像是學生的學號、名字、綽號、身分證字號、駕照號碼都可以代表同一個人。

物件 (OBJECT)

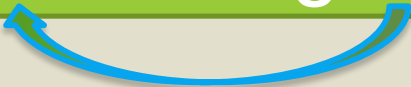
- **Python** 的世界裡，一切皆物件
每個物件各包含一個 **identity**、**type** 和 **value**。



賦值 (ASSIGNMENT)

- 賦值/指定
是將特定資料值指派給變數
- **data = 256**
- 上面這個運算式的意思就是將**256**這個數值賦予給**data**這個變數。
- 「**=**」不是數學邏輯，而是**賦值/指定**的意思！

Left value **=** right value



- 從這行程式執行之後，**data**的內容即裝著**256**，直到**data**被賦予其他值為止。

A decorative graphic on the left side of the slide consisting of two parallel, wavy vertical lines. The inner line is a vibrant lime green, and the outer line is a light cream color. They extend from the top to the bottom of the frame.

PYTHON DATA STRUCTURES

PART 3

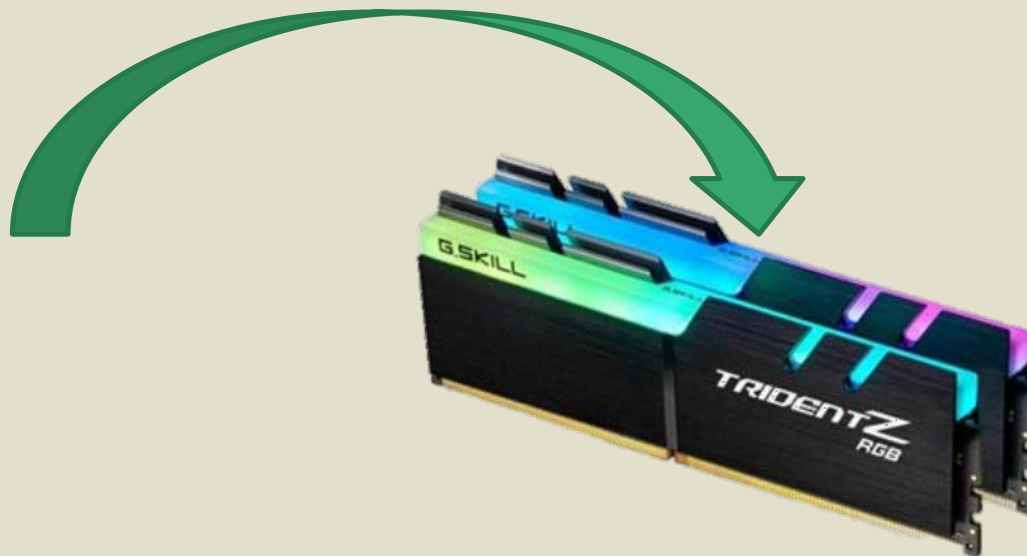
資料型態

- 在**Python** 的世界裡面，資料會以各種不同的型別儲存在電腦裡，以供**Python**程式進行存取
- 其中，資料型態主要可分成三種類型：

數值型別

字串型別

容器型別



- 這些資料型態就像是魔法世界裡的元素屬性一般，可以描述物件的不同性質，彼此之間卻又可以被搭配使用來完成特定的任務。

數值型別 (NUMERIC TYPE)

int, float, bool, complex

- **int** : **integer** 的縮寫，也就是**整數**的意思，負責儲存**沒有小數點**的整數數值。
- **float** : 浮點數，可以儲存**具有小數點**的數值。
- **bool** : **boolean** 的縮寫，中文通稱為「布林」，只有三種資料值，包括**True**、**False** 與**None**，分別代表**真**、**假**和**空值**三種意思。
- **complex** : 代表數學中的**虛數**，以**complex (x, y)** 的形式出現。其中，**x** 為**實部的數字**，而**y** 為**虛部的數字**。除此之外，也可以在數字後面直接加**j** 來代表虛數部，如：**1+5j**、**6.7-4.2j** 等。

字串型別 (STRING TYPE)

- 字串型別 (**String Type**) - **str**
- 在**Python** 裡面，各位可以用「**'**」，「**"**」，「**'''**」把文字包住，而這些都代表字串的意思。
- 字串有可以被拆解使用，且具有前後順序的特性，例如在字串變數**x="Hello, World"** 裡面，**x** 這個字串裡面有**12** 個字元。
依序是『**H**』、『**e**』、『**l**』、『**l**』、『**o**』、『**,**』、『』、『**W**』、『**o**』、『**r**』、『**l**』、及『**d**』。

容器型別 (CONTAINER TYPE)

- **tuple, list, set, dict**

- 這個型別裡面的資料就像是箱子一樣，可以裝入各種不同型態的資料，也可以在箱子裡裝入其他的箱子喔。

tuple：序對，在 **()** 裡面可以放置一個以上的資料值，有順序性，但是不能更改其內容。

list：串列，在 **[]** 裡面可以放置一個以上的資料值，是一種有順序且可以更改其內容的型態。

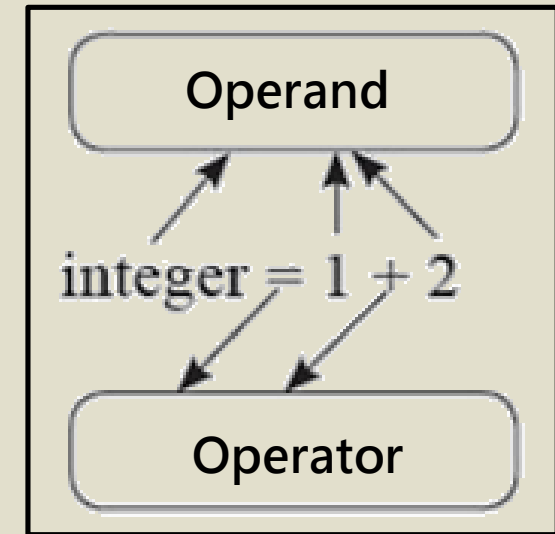
set：集合，在 **{ }** 裡面可以放置一個以上的資料值，類似數學裡面的集合概念，所以內容並無順序性。

dict：字典，是 **dictionary** 的縮寫，以 **Key-Value** 對應的型態在 **{ }** 裡面放置一個以上的元素。字典是種配對型別，也可以放置一個以上的資料值，而 **key** 就是用來存取每個 **value** 的索引值。

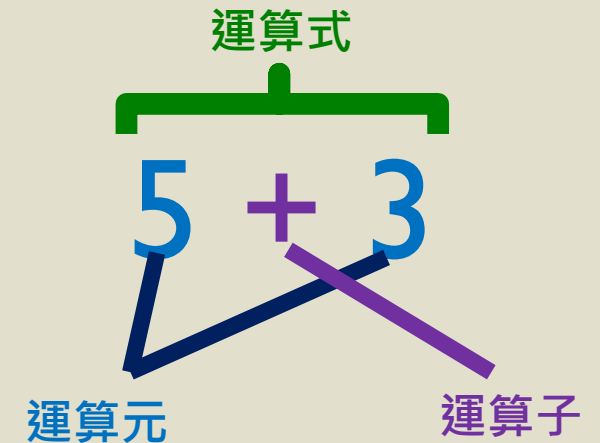
運算式 (EXPRESSION)

- Expression = Operator + Operand
- Operand 個數要符合Operator的要求
- Operand 的資料型態要一致

Expression



- 大部分的程式碼都是由判斷式及運算式組成的，但此時先不談判斷式，我們先了解一下運算式。
- $5 + 3 = 8$ 這種國小就有教過的式子就是運算式。
- 運算式是由運算子及運算元所組成，
- 運算子就像是這個運算的種類 (如+, = 等)
- 運算元則是要被用來運算的資料 (如1, 2, integer 等)



運算子 (OPERATOR)

- 運算子可以分成：

算術運算子

比較/關係運算子

指派運算子

邏輯運算子 (又稱布林運算子)

其他運算子

運算子 (OPERATOR) - 算術運算子

- 就是一般數學式中常用的加減乘除

運算子	意義
+	加法運算子，執行兩運算元之加法。
-	減法運算子，執行兩運算元之減法。
*	乘法運算子，執行兩運算元之乘法。
**	指數運算子。
/	除法運算子，執行兩運算元之除法。
//	整數除法運算子，出來的結果會自動取整數
%	模數運算子，取餘數用

運算子 (OPERATOR) - 比較/關係運算子

- 比較運算子又可稱為關係運算子，將兩個運算元拿來相互比較，以得知兩者之間的關係
- 「=」是賦值，「==」雙等號表示比較運算子
- 運算後所得出的結果則是布林值 (True/False)。

運算符號	意義
>	大於
<	小於
>=	大於或等於
<=	小於或等於
==	等於 ※ 和「=」（指定運算子）不同！
!=	不等於

Operation	Meaning
<	strictly less than
<=	less than or equal
>	strictly greater than
>=	greater than or equal
==	equal
!=	not equal
is	object identity
is not	negated object identity

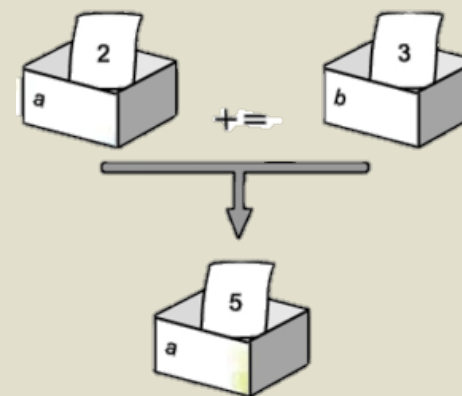
TRUE AND FALSE

- 對於任意對象，直接判斷其真假，無需寫判斷條件，既能保證正確性，又能減少代碼量

真	假
True	False
任意非空字符串	空的字符串 ""
任意非0数字	数字0
任意非空容器	空的容器 [] () {} set()
其他任意非False	None

運算子 (OPERATOR) – 指派運算子

- 指派運算子是將運算過後的結果儲存在某個變數中



- 使用複合指派運算子的好處是可以讓運算式變簡單

運算子	例子	意義
=	$a = b$	將b的值賦予a
+=	$a += b$	$a = a + b$
-=	$a -= b$	$a = a - b$
*=	$a *= b$	$a = a * b$
/=	$a /= b$	$a = a / b$
%=	$a \% = b$	$a = a \% b$

運算子 (OPERATOR) – 邏輯運算子

■ **Boolean Operations** → the result is still true or false

Operation	Result
<code>x or y</code>	if <code>x</code> is false, then <code>y</code> , else <code>x</code>
<code>x and y</code>	if <code>x</code> is false, then <code>x</code> , else <code>y</code>
<code>not x</code>	if <code>x</code> is false, then <code>True</code> , else <code>False</code>

運算	意義
<code>x or y</code>	若 <code>x</code> 為假，則傳回 <code>y</code> ；若 <code>x</code> 為真，則傳回 <code>x</code>
<code>x and y</code>	若 <code>x</code> 為假，則傳回 <code>x</code> ；若 <code>x</code> 為真，則傳回 <code>y</code>
<code>not x</code>	若 <code>x</code> 為假，則傳回 <code>True</code> ；若 <code>x</code> 為真，則傳回 <code>False</code>

運算子 (OPERATOR) – 邏輯運算子

■ **Boolean Operations → the result is still true or false**

運算子	意義
and	" 而且 ", 所有的條件都要成立, 整個判斷才會成立
or	" 或 ", 只要有任何一個條件成立, 整個判斷就會成立
not	" 非 ", 條件不成立時, 整個判斷就會成立

運算子 (OPERATOR) – 其他運算子

- 除了上述所提到的四大類運算子之外，還有一些比較少用，且無法歸類到上述四大類運算子之中，故另外以下表為各位介紹。

運算子名稱	運算子	意義
逗號運算子	,	分隔變數、資料集裡的元素等等
分號運算子	;	分隔運算式
點號運算子	.	存取類別、模組的方法或屬性
小括弧運算子	()	定義tuple、函式/方法呼叫、提升運算優先權
中括弧運算子	[]	定義list、序列形態的索引符號
大括弧運算子	{}	定義dict、set
冒號運算子	:	控制條件後的分隔符號或辭典元素之配對

運算子優先順序 (OPERATORS PRECEDENCE)

運算子	說明
(...)、[...]、{...}	tuple、list、dict
a[i]、a[i:j]、a.b、a (...)、a.b (...)	內含中小括號之呼叫
+k、-k、~k	正負數及補數
a * b、a / b、a % b、a // b	乘法、除法、取餘數、整數除法
a + b、a - b	加法、減法
a << b、a >> b	位移運算
a & b	AND位元運算
a ^ b	XOR位元運算
a b	OR位元運算
<、>、<=、>=、==、<>、!=、is、is not、in、not in	比較運算、本體測試、序列元素關係測試
not a	not邏輯運算
a and b	and邏輯運算
a or b	or邏輯運算

字典 (DICT)

- 字典 (**dict**) ，為**dictionary** 的縮寫
- 具有**Key-Value** 對應的型態
- 字典是種配對型態，且具有多筆資料的物件，**key** 就是存取該筆**value** 的索引值

D1 = { “Key” : Value , “Key” : Value } 共兩筆資料

- 同樣要建立一個**dict**，有四種可以用的方法：

```
d1 = {1: 'a', 2: 'b'}  
d2 = dict({1: 'a', 2: 'b'})  
d3 = dict(zip((1, 2), ( 'a', 'b')))  
d4 = dict([[2, 'b'], [1, 'a']])
```

序對(TUPLE)

- 序對 (**tuple**) 會在() 裡面放置資料，這些資料有順序但是不能更改。
- **tuple** 內的資料是從0 開始編號，也可使用[] 來取出特定index 值的資料
- 此外，Python 中有順序的資料型態 (如list、str 等) 的index 值也都可以這樣使用
- 換言之，如果t 不是空的，那第一個元素必定為t[0]
- 如果要找到**tuple** 裡面的最後一筆資料，可以使用負數索引
- 也就是，如果t 不是空的，則最後一個元素必定是t[-1]

陣列(LIST)

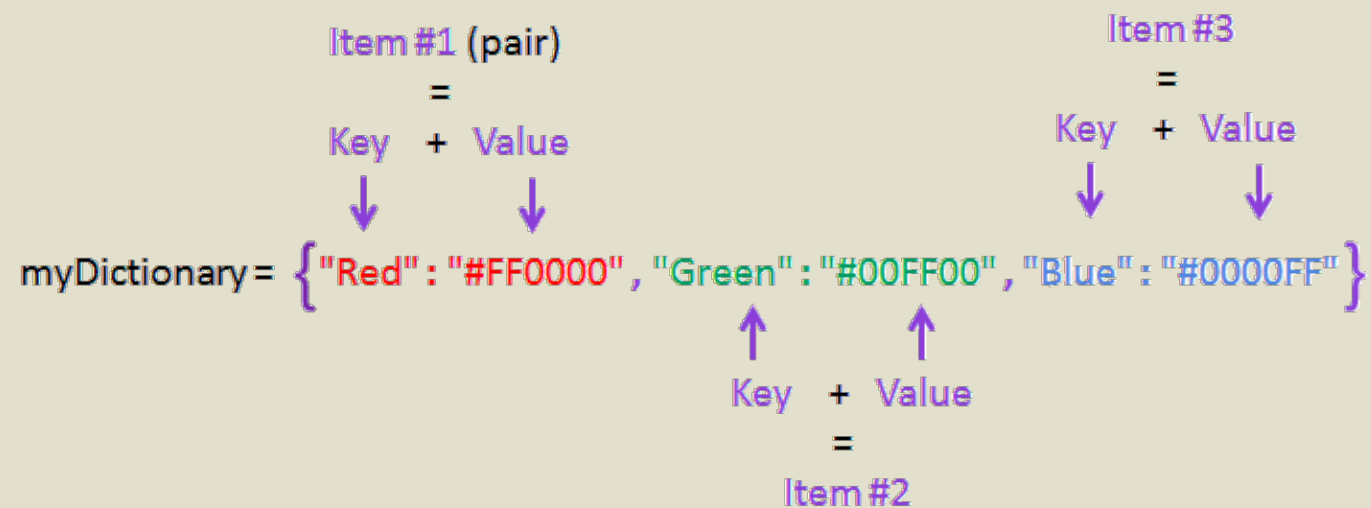
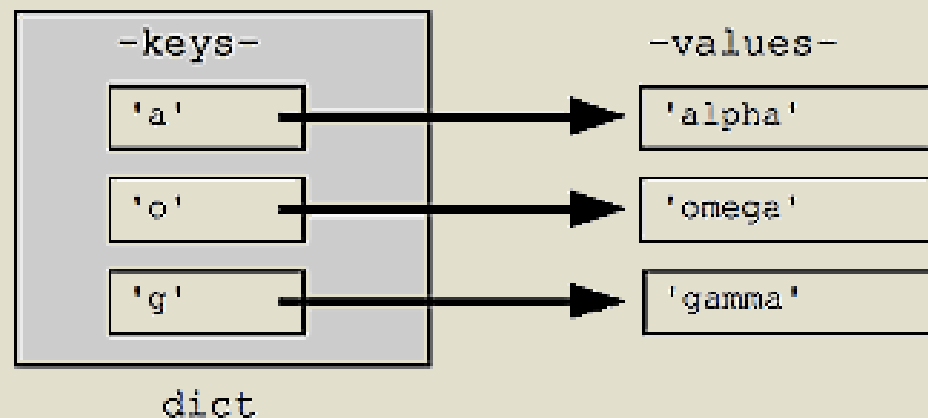
- 序對 (**tuple**) 會在() 裡面放置資料，這些資料有順序的，且具有可以更改的型態。
- 這裡列出一些常用的**list** 的函式：

函數	描述
list.append(object)	在list的最後面加入object。
list.extend(l)	將另一個具有順序的物件l附加在此list的最後面。
list.insert(index, object)	將元素object插入在index值為i的位置。
list1 = list2 + list3	將 2 個 List 的內容相加
list.remove(value)	將list裡面的第一個符合value之元素刪除。
list.pop(i)	將list裡面index值為i的元素刪除並回傳。 如果沒有給定i值，則刪除並回傳最後一個

函數	描述
list.clear()	將整個list清空。
list.index(value, [start, [stop]])	查詢list當中第一個value出現的index值並回傳。
list.count(value)	計算list中value出現的次數。
list.sort()	將list裡面的元素做排序並儲存。
list.reverse()	將list裡面的元素順序顛倒並儲存。
list.copy()	將list直接複製一份。

字典 (DICT)

- 字典語法執行示意圖



字典 (DICT)

- **dict**的讀取、刪除、回傳與判斷等方式則如下面內容所示。

計算	描述
<code>d[x]</code>	從d中取得x所對應的值
<code>d[x]=y</code>	將d中x所對應的值指定為y 若d中沒有x這個key則新增一組
<code>del d[x]</code>	刪除d中x所屬的組合
<code>x in d</code>	判斷x是否在d的key值中
<code>x not in d</code>	判斷x是否不在d的key值中
<code>iter(d)</code>	回傳由d的key值所建立的迭代器
<code>len(d)</code>	回傳d的資料組數
<code>dict.clear()</code>	
<code>dict.copy()</code>	
<code>dict.pop()</code>	<code>pop()</code> 要放入key值
<code>dict.keys()</code>	
<code>dict.values()</code>	

集合 (SET)

- 集合會在`{ }`裡面放置元素，類似數學裡面的集合概念
- 由於集合是可變動的，有幾個方法是可以對**set**作更動的
- 下表將**set**的函式分別列出作介紹：

函數	功能
<code>set.pop()</code>	將set的隨意一個元素回傳給呼叫的程式碼之後將此元素刪除。
<code>set.add(e)</code>	將元素e加入set。
<code>set.remove(e)</code>	將元素e從set中移除。
<code>set.clear()</code>	將set的所有元素清空。
<code>set.copy()</code>	回傳一份複製的set。
<code>set.discard(e)</code>	將e從set中捨棄。
<code>set.difference(set2) ; set - set2</code>	將兩個set作差集運算，並回傳一個新的set
<code>set.difference_update(set2)</code>	將兩個set作差集運算，並將結果更新到set

集合 (SET)

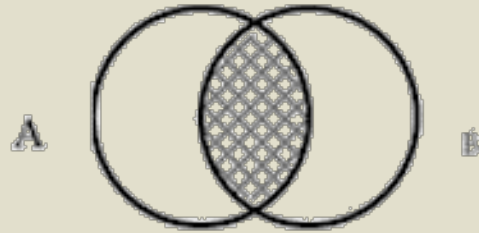
- 集合的函式

函數	功能
<code>set.intersection(set2) ; set & set2</code>	將兩個set作交集運算，並回傳一個新的set
<code>set.intersection_update(set2)</code>	將兩個set作交集運算，並將結果更新到set
<code>set.union(set2) ; set set2</code>	將兩個set作聯集運算，並回傳一個新的set
<code>set.update(set2)</code>	將兩個set作聯集運算，並將結果更新到set
<code>s.symmetric_difference(t) ; set ^ set2</code>	將兩個set作互異運算，並回傳一個新的set
<code>s.symmetric_difference_update(t)</code>	將兩個set作互異運算，並將結果更新到set
<code>set.issubset(set2) ; set <= set2</code>	判斷set是否為set2的子集合，回傳bool值
<code>set.issuperset(set2) ; set >= set2</code>	判斷set2是否為set的子集合，回傳bool值

集合 (SET)

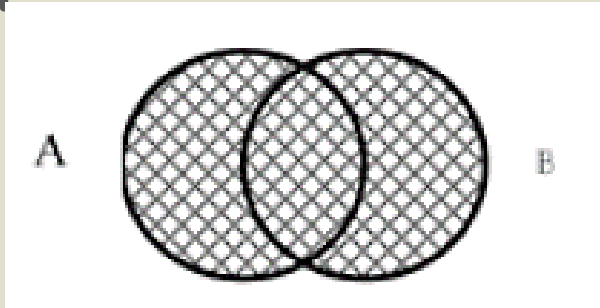
- 交集 and intersection

$A \cap B$



- 聯集 or union

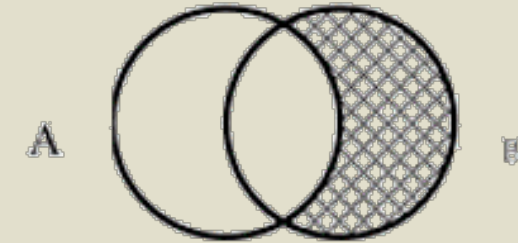
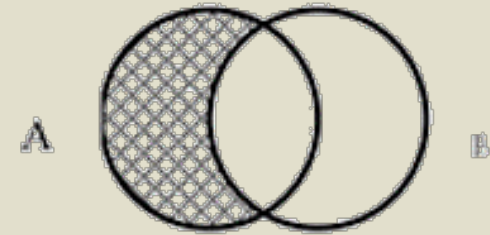
$A \cup B$



- 差集 difference

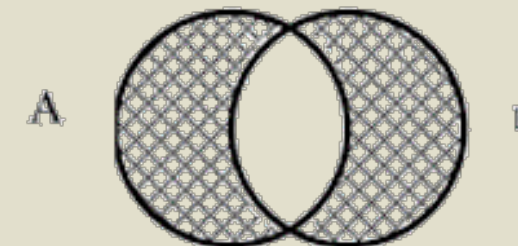
$A - B$

$B - A$



- 互斥(異)或 symmetric_difference

$A \oplus B$



A decorative wavy line in a light green color runs vertically along the left side of the slide, separating the title area from the rest of the page.

PYTHON PROGRAMMING FUNDAMENTALS

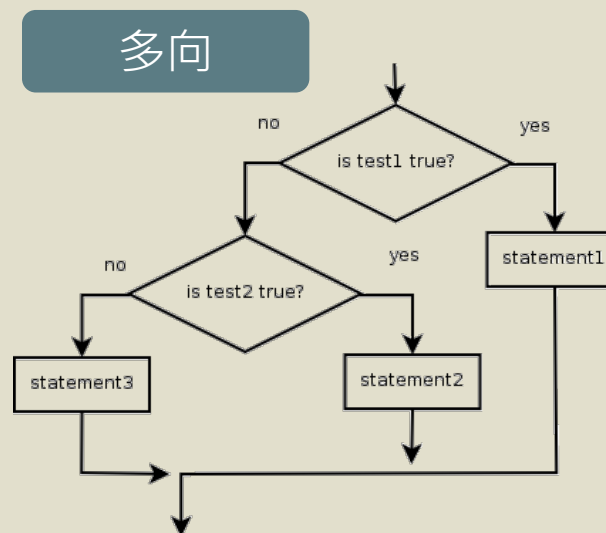
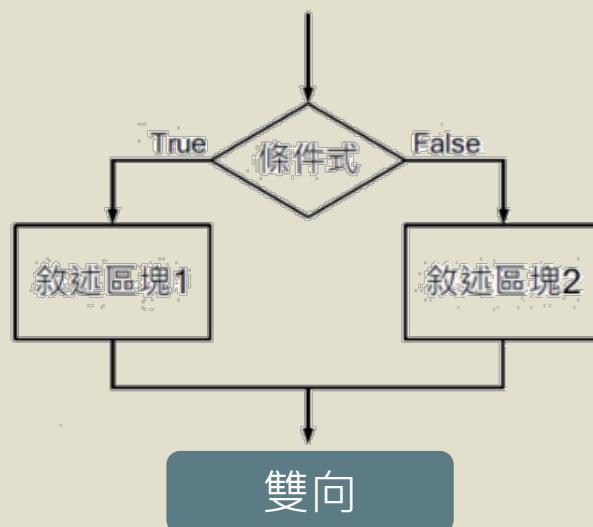
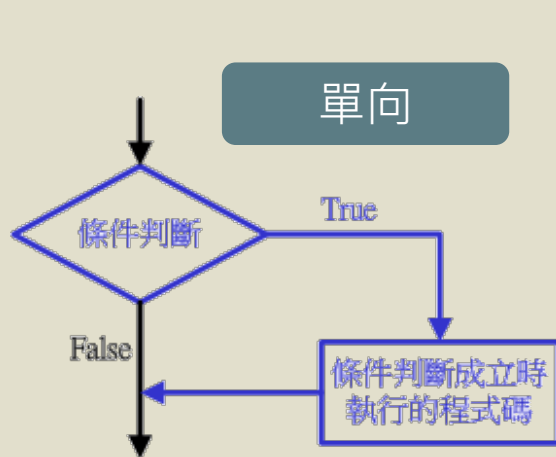
PART 4

判斷式 (CONDITIONAL STATEMENT)

- **Python** 是依照甚麼樣的方式在執程式碼
- 在沒有特殊情形之下，會從程式碼的第一行開始執行，依序向下
- 特殊情形有以下四種：
 - 判斷式
 - 迴圈
 - 函式/函數
 - 例外

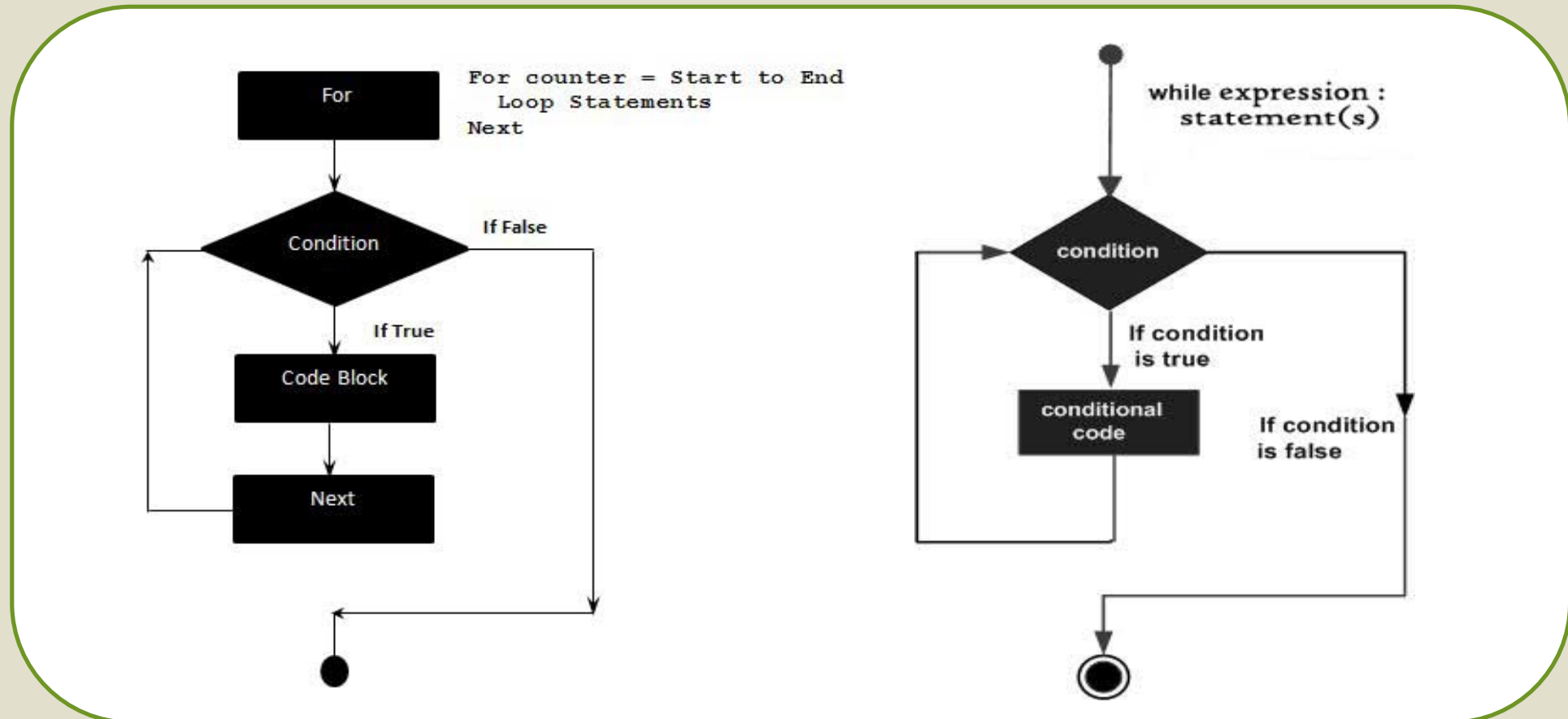
判斷式

- 當程式執行到if 判斷式時，判斷現在的情形是否符合撰寫所設定的條件
- 所以可能會因條件不符合，而發生跳過不執行某些區塊程式碼的情形



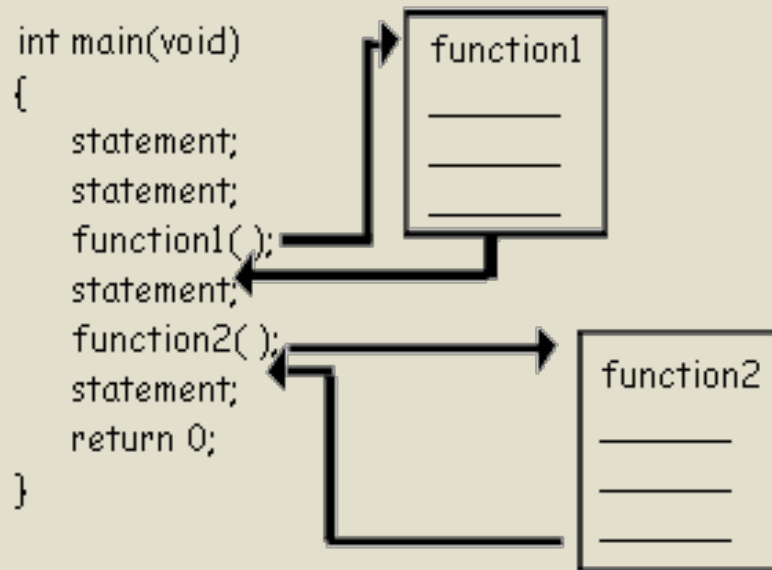
迴圈

- 迴圈內的程式碼需要重複執行的程式碼有for 迴圈跟while迴圈兩種類型
- 當程式碼執行到迴圈時，會根據程式撰寫時所設定的條件作判斷，若符合則重複執行迴圈裡面的程式碼



函式/函數

- 函式是為了完成特定功能而被撰寫的程式碼，有可以被重複呼叫的特性。
- 當程式執行時遇到呼叫函式，會去尋找此函式的完整程式碼，然後利用程式呼叫函式時所傳入的參數，進而執行該函式的程式碼。
- 在這種情形下，通常會產生跳躍式執行程式碼的可能性。



```
def fahr_to_celsius(temp):
    return ((temp - 32) * (5/9))
```

The diagram labels the components of the function definition: `def` is the **def keyword**, `fahr_to_celsius` is the **name**, `(temp)` is the **parameter**, `return` is the **return statement**, and `((temp - 32) * (5/9))` is the **return value**.

函式/函數

- 呼叫函式的基本方法

Funtion_name (Arg1, Arg2, ...)

命名的方法如同變數一樣

給予引數

- 函式名稱後面跟隨著小括號
範例：**print (“ Hello, world”)**
- 內建函式可以[直接使用](#)
- 內建函式:**dir(__builtins__)**、**iterator: range()**、**Length: len()**
- 轉換型態或建立物件 **int()**、**str()**、**float()**、**list()**、**set()...**

函式/函數 - 內建

- 內建函式(**Build-in function**)

內建函數 (Build-in Function)				
Built-in	Functions			
abs()	dict()	help()	min()	setattr()
all()	dir()	hex()	next()	slice()
any()	divmod()	id()	object()	sorted()
ascii()	enumerate()	input()	oct()	staticmethod()
bin()	eval()	int()	open()	str()
bool()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	
delattr()	hash()	memoryview()	set()	

函式/函數 - 自訂

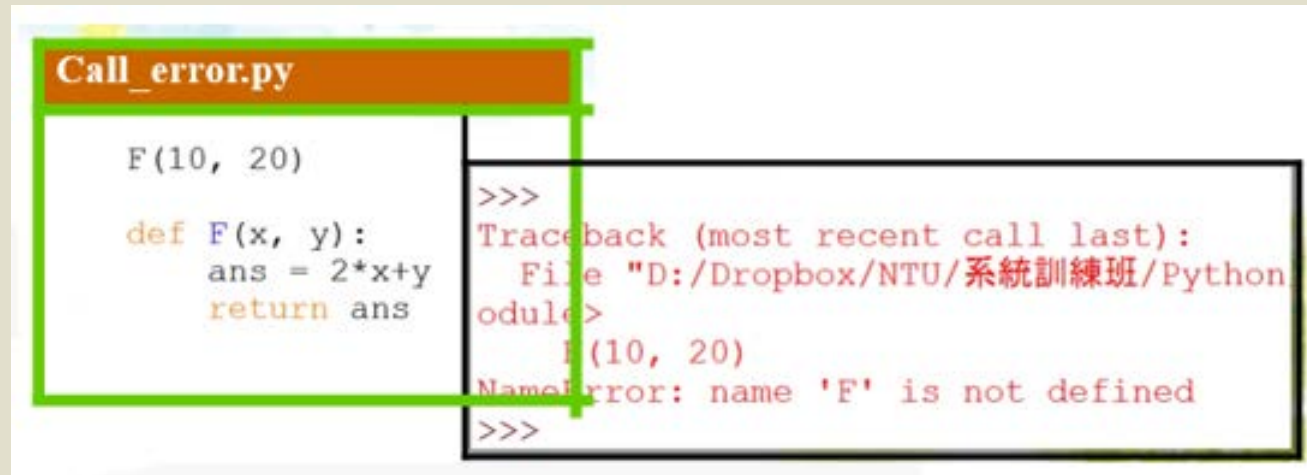
- 包函許多程式碼的一行程式 (用來代表某種功能)
- 當程式碼太多且會重覆出現時，可以將部份程式碼抽離主程式，寫成一段函式，有需要用到時再去呼叫它
- 函式是經過組織且可重複使用的程式碼，是能用來實現單一或是相關聯的程式碼
- 巧妙的運用函式可以提高程式碼的重複利用率，避免一樣的事情卻需要重複寫好幾次的程式碼來執行。
- 這跟迴圈的概念有點像，都是在重複利用程式碼；不同的地方在於函式是在需要時才呼叫使用

函式/函數 - 參數與引數

- 參數與引數的差別(很容易混淆)
- **Parameter** / 參數 (虛引數)
函式運行時之參考
放在函式的標記式，用來說明這個函式被呼叫時必須[接收](#)到什麼樣的資料
- **Argument** / 引數 (實引數)
用以呼叫引發函式
呼叫函式時，放在括號內[傳入](#)被呼叫的函式

函式/函數 - 傳入參數回傳值

- 注意先有函數定義，才可進行函數呼叫



```
Call_error.py

F(10, 20)

def F(x, y):
    ans = 2*x+y
    return ans

>>>
Traceback (most recent call last):
  File "D:/Dropbox/NTU/系統訓練班/Python
odule>
    F(10, 20)
NameError: name 'F' is not defined
>>>
```

- 因為 **Python** 直譯器從頭一行一行的解譯程式原始碼 (**source code**)，**F()** 既非 **Python** 的內建名稱，也還沒解譯到底下定義的部份，因此直譯器直接發生 **NameError**，終止程式的進行

函式/函數 - 預設參數

- **The default value is evaluated only once.**
- This makes a difference when the default is a **mutable** object such as a list, dictionary, or instances of most classes.

```
def f1(a, L=[]):  
    L.append(a)  
    return L  
  
def f2(a, L=None):  
    if L is None:  
        L = []  
    L.append(a)  
    return L  
  
print(f2(1))  
print(f2(2))  
print(f2(3))
```

函式/函數 - 不定個數參數

- 函數 (function) 可以有不定個數的參數(parameter) ，也就是可以在參數列(parameter list) 提供任意長度的參數個數

Multi_arg.py

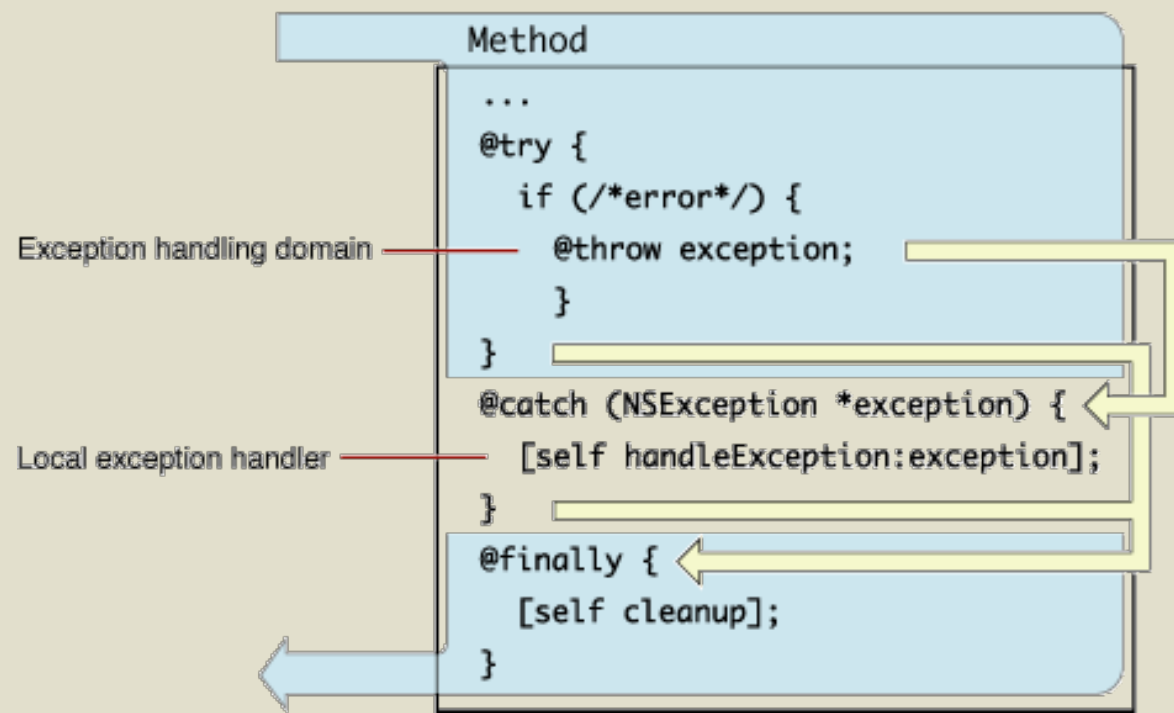
```
1 def function_name(*arguments, **keywords):  
2     #dosomething  
3     return something
```

*arguments 就是參數識別字 (identifier) 前面加上一個星號，當成一組序對 (tuple)

**keywords 參數識別字前面加上兩個星號，當成一組字典 (dictionary)

例外

- 這裡所說的例外就是錯誤，例如縮排不一致、格式發生衝突等等。



- 出現這種情形如果沒有妥善處理，程式的執行就會強制中斷

縮排

- 以上四種特殊情形會改變從上至下逐行檢查並執行的動作，可以完成更複雜的程式結構與功能，讓所撰寫出來的程式能完成更複雜的動作。
- Python 在程式區塊中強制使用縮排，而不使用{}
令程式碼更簡潔，增加可讀性
在 Java/C/C++ 中縮排可做可不做
但在 Python 中，一定要做縮排
- 如果縮排沒有對齊，那程式碼是無法執行的
Python 會跟你說出現預期以外的縮排行為

```
# Prints a helpful message.  
def hello():  
    print("Hello, world!")  
    print("How are you?")  
  
# main (calls hello twice)  
hello()  
hello()
```

列表推導式

- 在list中存入0~10之間3的的倍數平方值
- 一般寫法

```
1 l = []  
2 for x in range(10):  
3     if x % 3 == 0:  
4         l.append(x*x)  
5 #l = [0, 9, 36, 81]
```

- 推導式寫法

```
1 l = [x*x for x in range(10) if x % 3 == 0]  
2 #l = [0, 9, 36, 81]
```

ENUMERATE

- `enumerate`可以一次性將索引值和該位置儲存的數值取出
避免使用索引來取值，第二個參數為索引的起始位置，預設為0

- 一般寫法

```
1 array = [1, 2, 3, 4, 5]
2
3 for i in range(len(array)):
4     print(i, array[i])
```

- `enumerate`寫法

```
1 array = [1, 2, 3, 4, 5]
2
3 for i, e in enumerate(array, 0):
4     print(i, e)
```

A decorative wavy line in a light green color runs vertically along the left side of the slide, separating the dark green background from a light beige area.

WORKING WITH DATA IN PYTHON

PART 5

模組(MODULES)

- 模組(**Modules**) 可以組織 **Python** 程式碼，將相關程式碼集合在一個模組可以讓程式更容易了解與使用
- 模組視為一個 **Python** 物件，可以給模組一個名稱，讓程式來使用該模組的內容
- 簡言之，模組是一個檔案，內含 **Python** 程式碼。模組可以定義函數(**Functions**)、類別(**Classes**) 以及變數，也可以包含執行碼(**Runnable codes**)

模組(MODULES)

- 其實模組就是.py 檔，之前做過的練習儲存起來的.py 檔都是模組
- 而當各位需要使用到的時候，都可以藉由import指令來匯入現在所編輯的程式中使用。
- 至於之前所import 的模組，多是Python內建的模組，以方便大部分程式設計人員所使用，而不用一再重新撰寫
- 模組就是保存了程式碼的.py 檔，裡面可能定義了函數、類別、變數等內容

模組(MODULES)

- 要匯入模組，則有兩個指令可以加以使用：import跟from...import，其語法分別是

```
import module1[,module2[,...module]
```

```
From modname import name1[,name2[,...nameN]]
```

- import 可以一次匯入多個模組，中間以「，」分隔
- 而from...import 則是更精確的描述想要使用的是哪個模組裡面的哪個函數
- 如果匯入模組檔案的路徑底下找不到你所指定的檔案，那Python 便會到 `sys.path` 裡面尋找

模組

- 透過上面的範例便可知道Python 會在哪裡系統的路徑下找尋要import 的模組檔案
- 如果import 的模組名稱太長，或是想要換名稱，則可使用import...as 這個指令

```
01 import math as ma      # import math 模組，並將其重新命名為ma
02 print(ma.pi)           # 輸出ma 模組裡的變數pi
```

```
>>>
3.141592653589793
>>>
```

- 使用import...as 並不會改變這個模組真正的名字，僅是在被匯入的這支程式裡面被更改名稱而已

模組 - 好用的標準函式庫TIME

- **time.time()** → 取得系統時間
- **time.sleep(num)** → 設定暫停時間
- **time.localtime()**

回傳格式：

**time.struct_time(tm_year, tm_mon, tm_mday,
tm_hour, tm_min, tm_sec, tm_wday, tm_yday_, tm_isdst)**

- **time.strftime(TimeFormat)** → 格式化輸出時間
- **time.gmtime()** → 取得 **UTC** 世界標準時間

#TIME 模組

- 請使用系統內建time模組 存檔成(MyTime.py)

定義一函式GetTime可回傳呼叫/現在時間

令呼叫函式可自定輸出時間格式

```
>>>help(time.strftime)
```

Help on **built-in function** strftime in module time:

strftime(...)

strftime(format[, tuple]) -> string

Convert a **time tuple** to a **string** according to a **format specification**.

See the library reference manual for formatting codes. When the time tuple is not present, **current time** as returned by **localtime()** is used.

- Ex :

```
GetTime( '%Y-%m-%d %H:%M:%S' )
```

Hint : 使用time.strftime()

#TIME模組

Commonly used format codes:

%Y Year with century as a decimal number.
%m Month as a decimal number [01,12].
%d Day of the month as a decimal number [01,31].
%H Hour (24-hour clock) as a decimal number [00,23].
%M Minute as a decimal number [00,59].
%S Second as a decimal number [00,61].
%Z Time zone offset from UTC.
%a Locale's abbreviated weekday name.
%A Locale's full weekday name.
%b Locale's abbreviated month name.
%B Locale's full month name.
%c Locale's appropriate date and time representation.
%I Hour (12-hour clock) as a decimal number [01,12].
%p Locale's equivalent of either AM or PM.

#TIME 模組

- 自訂模組MyTime.py



```
import time

t1 = time.time()

def GetTime(fmt):
    loc1 = time.localtime(t1)
    return time.strftime(fmt, loc1)

if __name__ == '__main__':
    print(GetTime("%Y-%m-%d"))
    print(GetTime("%Y-%m-%d %H:%M:%S"))
    print(GetTime("%Y-%m-%d %b %B %H:%M:%S %a %A"))
```

- 引入剛剛的小練習
呼叫GetTime()函式

01	import MyTime
02	
03	print(" 現在時間: " + MyTime.GetTime('%Y-%m-%d %H:%M:%S'))

>>>

現在時間: 2014-06-21 17:43:54

>>>

檔案與資料夾管理模組

os模組：顯示系統環境參數與指令功能函數

- `os.chdir(path)`與`os.getcwd()`

`os.chdir(path)`函數是切換目錄到path引數位置

`os.getcwd()`是顯示目前所在的目錄位置

- `os.mkdir(path[, mode])`與`os.rmdir(path)`

`os.mkdir()`是建立資料夾

path引數是建立/刪除目錄的位置

mode引數是Unix平台使用的

`os.rmdir()`函數是刪除目錄

- `os.path.getsize(path)` → 取得檔案大小
- `os.path.getctime(path)` → 取得檔案的建立日期

檔案與資料夾管理模組

- `os.path.getmtime(path)` → 取得檔案的修改日期
- `os.path.getatime(pah)` → 取得檔案的存取日期
- `os.path.isfile(path)` → 判斷傳入的path引數是否為檔案
- `os.path.isdir(path)` → 判斷傳入的path引數是否為目錄
- `os.path.exists(path)`
判斷傳入的path引數目錄或檔案是否存在
- `os.system("...")` #`os.system("pause")`
執行 Command line 命令(根據作業系統會不一樣)

檔案與資料夾管理模組

- `os.rename(src, dst)` → 對檔案或目錄更換名稱
src引數是原本的資料夾
dst引數是修改後的資料夾名稱
- `os.remove(path)` → 移除檔案
path引數傳入檔案位置
不會移除資料夾，若要移除資料夾可用下一個指令
- `os.removedirs(path)` 或 `os.rmdir(path)`
移除空的資料夾
- `os.listdir(path)` path可不寫
輸出path引數位置的目錄和檔案名稱 *預設當前路徑

SHUTIL 模組

- 使用shutil模組 - High-level file operations
比較高階的應用層，提供數個針對檔案操作的功能

- `shutil.copytree(src, dst)`
複製整個目錄，包含目錄內的所有檔案
- `shutil.copy(src, dst)` → 單純複製檔案
- `shutil.rmtree(path)`
移除整個目錄，包含目錄內的所有檔案
- `shutil.move(src, dst)`
移動檔案，移動時也可以進行更換檔案名稱
- `shutil.copystat(src, dst)`
複製檔案，會連同檔案屬性一同複製

常用的套件簡介

- 列出一些有名、常用到的或是有趣的第三方套件，這些套件可以單獨使用或是配合Django來完成強大的功能。

- 網站框架：

Django：完整強大的Web框架

Pyramid：另一個完整強大的Web框架

web2py：Google app engine 預設使用的框架

flask：相較而言是輕量的網站框架

常用的套件簡介

- 影像、圖片處理：

PIL : Python Imaging Library，PIL 是Python 下最有名的影像處理套件，由許多不同的模組所組成，並且提供了許多的處理功能，允許我們在簡單的Python 程式裡進行影像的處理。

Pillow : PIL的fork版本，另一套圖形處理的套件。

OpenCV: 知名影像處理函式庫的Python 版本。

- 科學計算：

Numpy：支援非常多的科學計算，包含矩陣運算、線性代數、傅立葉轉換等。大多數的科學計算領域上都派得上用場。

Scipy：提供最佳化、線性代數、訊號處理和圖像處理、常微分方程求解...和其他科學與工程中常用的計算等功能

Matplotlib：可以畫出各種圖形如長條圖、分布圖、立體圖等...

pandas：處理特殊資料結構，具有數據處理與資料分析的功能

scikit-learn：機器學習的套件，包含內建的分群分類計算、回歸、統計等功能

常用的套件簡介

- 命令列操作及遠端登入：

fabric：可以直接撰寫shell命令，透過fabric執行，也支援遠端登入和自定義shell命令。

paramiko：提供遠端登入和部分指令呼叫的功能。

- 網路爬蟲：

Scrapy：Python爬蟲框架，可以輕易的與Django合作。

Requests：發送網路請求，抓取網頁文件。

- 文件解析器：

beautifulsoup：可以處理HTML、XML等標記格式。

lxml：可以處理HTML、XML等標記格式，使用xpath選取內容。

檔案處理

- 從檔案讀進資料或將結果存入檔案之中

- 從檔案讀進資料

`file.read()`

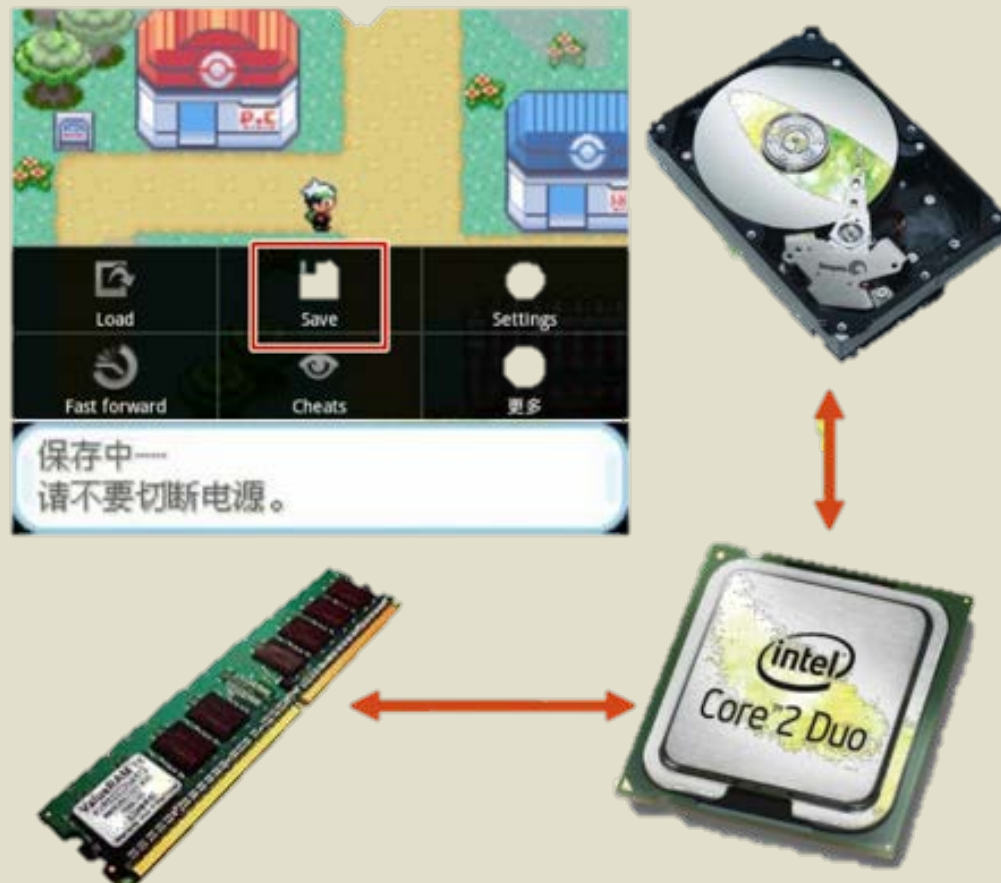
`file.readline()`

`file.readlines()`

- 將結果存入檔案

`file.write()`

`file.writelines()`



檔案處理

- **open()** 函數來開啟電腦中已經存在的檔案，以便能進行檔案內容的讀取、寫入或修改等動作



- 使用方式如下所示：

```
file object = open(file_name [, access_mode][, buffering] )
```

檔案處理

```
file object = open(file_name [, access_mode][, buffering] )
```

- **[]** 裡面的參數都是可省略的
- **file_name** 就是想開啟的檔案，其中也**包含了檔案的路徑**，如果沒有包含路徑的話，則要開啟的檔案與目前在編輯的程式碼檔都位於**同一個資料夾**
- **access_mode** 想要存取的模式，種類繁多，下面分別列表說明
- **buffering** 資料讀入的暫存空間
0 代表沒有暫存空間，大於或等於**1** 則是代表有暫存空間，及一次可讀入多少資料量。至於**-1**，則表示會用預設的暫存區大小來暫存所讀入的資料內容。

檔案處理

模式	描述
r	以唯讀方式打開文件。文件游標將會放在文件的開頭。預設模式。
r+	用於讀寫。文件游標將會放在文件的開頭。
w	功能只限定於寫入。如果該文件已存在則將其覆蓋。如果該文件不存在，創建新文件。
w+	能夠讀寫。如果該文件已存在則將其覆蓋。如果該文件不存在，創建新文件。
a	功能限定為追加資料。文件游標將會放在文件的結尾，新的內容將會被寫入到已有內容之後。如果該文件不存在，創建新文件進行寫入。
a+	能夠讀寫。文件游標將會放在文件的結尾。文件打開時會是追加模式。如果該文件不存在，創建新文件用於讀寫。

檔案處理

模式	描述
rb	以唯讀方式打開文件(二進制格式)。文件游標將會放在文件的開頭。預設模式。
rb+	以二進制格式打開一個文件用於讀寫。文件游標將會放在文件的開頭。
wb	(二進位格式)功能只限定於寫入。如果該文件已存在則將其覆蓋。如果該文件不存在，創建新文件。
wb+	(以二進位格式)能夠讀寫。如果該文件已存在則將其覆蓋。如果該文件不存在，創建新文件。
ab	(以二進位格式且功能限定為追加資料。如果該文件已存在，文件游標將會放在文件的結尾。也就是說，新的內容將會被寫入到已有內容之後。如果該文件不存在，創建新文件進行寫入。
ab+	(以二進位格式)功能限定為追加資料。如果該文件已存在，文件游標將會放在文件的結尾。如果該文件不存在，創建新文件並且能夠讀寫。

檔案處理

- 用**open** 開檔之後要執行的動作至少有以下三種：
讀檔，寫檔及關檔，以**read()**、**write()**、**close()** 為代表
讀檔有**read()**，**readline()**，**readlines()** 三種
寫檔有**write()**、**writeline()**、**writelines()** 三種
關檔就有**close()** 一種
- **read()** 一次可以讀取整份文件，通常會將整份文件的文字放在一個**string** 裡面，對於文件中「行」的概念處理起來相當麻煩，因此在讀檔時並不常使用
- **readlines()** 一次讀取整份文件後將文件依行為單位存成**list**，接下來就可以使用**for** 迴圈進行處理
- **readline()** 一次只讀取一行，會比較慢。如果記憶體的空間足夠，建議使用**readlines()**

檔案路徑

- 相對路徑

相對於現在目錄的路徑表示法，因此「**相對路徑**」所指到的檔案或目錄，**會隨著現在目錄的不同而改變**

*假設當前目錄於**D:\test\file**

./ 表示當前路徑，相當於**D:\test\file**

../ 表示當前路徑的上一級路徑，相當於**D:\test**

./data 表示當前路徑下一級路徑的**data**資料夾裡，相當於**D:\test\file\data**

- 絕對路徑

指的是一個**絕對的位置**，並不會隨著現在目錄的改變而改變，**EX：D:\test、D:\test\file、D:\test\file\data**

Or D:\\test

編碼

- 繁體中文自1980年代以來通常使用Big5中文編碼
- 簡體中文，傳統上使用GB2312、GBK或GB18030編碼
- **UTF-8 (8-bit Unicode Transformation Format)** 是一種針對Unicode的可變長度字元編碼，也是一種字首碼。它可以用來表示Unicode標準中的任何字元，且其編碼中的第一個位元組仍與ASCII相容，這使得原來處理ASCII字元的軟體無須或只須做少部份修改，即可繼續使用。因此，它逐漸成為電子郵件、網頁及其他儲存或傳送文字的應用中，優先採用的編碼。

CSV檔

- **Comma-Separated Values (逗號分隔值)**
- 檔案以純文字形式儲存表格資料（數字和文字）
- 一種被分隔的資料格式，被**逗號字元**分隔的**欄位/列**和以**換行結束**的**記錄/行**。
- 由任意數目的記錄組成，每條記錄由欄位組成，欄位間的分隔符號，最常見的是逗號或制表符號
- **CSV**檔案格式的通用標準並不存在，但是**7-bit ASCII**是最基本的通用編碼。

PROCESS OTHER DATA

- 用於序列化的兩個模組

json：用於字串和**Python**資料型別間進行轉換

pickle：用於**python**特有的型別和**python**的資料型別間進行轉換

PROCESS JSON DATA

- JSON (JavaScript Object Notation)

```
1 import json
2
3 data = {
4     'name' : 'ACME',
5     'shares' : 100,
6     'price' : 542.23
7 }
8
9 json_str = json.dumps(data)
10
11 print(json_str)
12
13
14 # Writing JSON data
15 with open('data.json', 'w') as f:
16     json.dump(data, f)
17
18 # Reading data back
19 with open('data.json', 'r') as f:
20     data = json.load(f)
21
22 print(data)
```

{"name": "ACME", "shares": 100, "price": 542.23}
{'name': 'ACME', 'shares': 100, 'price': 542.23}

PICKLE

- **Pickle**模組中常用函式

1. **pickle.dump(obj, file, [,protocol])**
將**obj**物件序列化存入已經開啟的**file**中
2. **Pickle.load(file)**
將**file**中的物件序列化讀出
3. **pickle.dumps(obj[, protocol])**
將**obj**物件序列化為**string**形式，而不是存入檔案中
4. **pickle.loads(string)**
從**string**中讀出序列化前的**obj**物件

PROCESS PICKLE DATA - WRITE

- Pickle: Python-specific data format (compact binary representation)
- <https://docs.python.org/3/library/pickle.html#module-pickle>

```
import pickle

# take user input to take the amount of data
number_of_data = 5
data = []

# take input of the data
for i in range(number_of_data):
    raw = input()
    data.append(raw)

# open a file, where you want to store the data
file = open('important', 'wb')

# dump information to that file
pickle.dump(data, file)

# close the file
file.close()
```

PROCESS PICKLE DATA - READ

```
22
23 # open a file, where you stored the pickled data
24 file = open('important', 'rb')
25
26 # dump information to that file
27 data = pickle.load(file)
28
29 # close the file
30 file.close()
31
32 print('Showing the pickled data:')
33
34 cnt = 0
35 for item in data:
36     print('The data ', cnt, ' is : ', item)
37     cnt += 1
```

Showing the pickled data:

The data 0 is : 0

The data 1 is : 1

The data 2 is : 2

The data 3 is : 3

The data 4 is : 4

REFERENCES

- Python 3 文件
 - <https://docs.python.org/3/index.html>
- The Python 教學
 - <https://docs.python.org/3/tutorial/index.html>
 - <https://cs231n.github.io/python-numpy-tutorial/#images>
- Realpython *video
 - <https://realpython.com/>
- GeeksforGeeks *example code
 - <https://www.geeksforgeeks.org/python-programming-language/>
- Python 10 minutes a day
 - <https://python-10-minutes-a-day.rocks/>